Katas de PLN — Edición extendida

Tokenización, Semántica Vectorial, Embeddings y Transformers — teoría, práctica y rúbricas

Raúl Andrés Alzate Gómez

2026

Katas de PLN — Tokenización, Semántica Vectorial y Embeddings

- Cómo usar este documento
- Bloque A — Tokenización subpalabra
  - Kata 01 — Byte-Pair Encoding (BPE) desde cero
  - Kata 02 — Comparar 4 familias de tokenizador con SentencePiece
- Bloque B — Semántica vectorial clásica (representación dispersa)
  - Kata 03 — VSM + TF-IDF + similitud coseno sobre Shakespeare
  - Kata 04 — Normalización L2: por qué importa (y cuándo no)
  - Kata 05 — Reducción de dimensión: PCA vs t-SNE
- Bloque C — Embeddings densos
  - Kata 06 — Word2Vec Skip-gram con Negative Sampling, desde cero
  - Kata 07 — GloVe vs fastText: el problema OOV y la morfología
  - Kata 08 — Clasificación: TF-IDF + Regresión Logística vs embeddings promediados
- Bloque D — Embeddings contextuales y de oración
  - Kata 09 — Polisemia con BERT/BETO: "banco" no es un solo vector
  - Kata 10 — Búsqueda semántica con SBERT / BioBERT
- Bloque E — Del RNN al Transformer (la arquitectura por debajo)
  - La historia en una página: el predecesor de los Transformers
  - Self-attention en una fórmula (la usarás en las katas 12–13)
  - Kata 11 — RNN/LSTM de carácter desde cero: ver el cuello de botella
  - Kata 12 — Atención de Bahdanau/Luong desde cero: el puente
  - Kata 13 — Self-attention escalada + multi-head desde cero
  - Kata 14 — Codificación posicional sinusoidal
  - Kata 15 — Inspeccionar y usar un Transformer real (HuggingFace)
- Cierre — Mapa de competencias y orden sugerido
  - El hilo conductor (para la reflexión final)
  - Criterio global de "kata terminada"

Katas de PLN — Tokenización, Semántica Vectorial y Embeddings

Edición extendida. Quince ejercicios autocontenidos con teoría formal, ejemplos numéricos resueltos a mano, código completo y ejecutable, salida esperada, errores comunes, variantes, rúbrica de evaluación y referencias.

Cómo usar este documento

> ***Qué es una kata.*** *Un ejercicio corto y repetible que entrena* ***una sola*** *competencia hasta automatizarla. El nombre viene de las artes marciales: una secuencia de movimientos que se repite hasta que el cuerpo la ejecuta sin pensar. En programación la popularizó Dave Thomas ("code kata"): no buscas terminar, buscas fluidez.*
>
> ***Por qué katas y no un proyecto grande.*** *Un proyecto mezcla diez problemas a la vez; cuando algo falla no sabes cuál de los diez conceptos fue. La kata aísla* ***un*** *concepto, con* ***un*** *criterio de éxito, para que el aprendizaje sea atribuible.*

Reglas de juego:

1. **Una kata de punta a punta antes de la siguiente.** Teoría → ejemplo a mano → código corriendo → reflexión escrita. Nada de saltar.
2. **Primero a mano, luego a máquina.** Cada kata trae un ejemplo numérico minúsculo que debes poder resolver con lápiz. Si no entiendes el ejemplo de 3 documentos, el de 3000 tampoco lo entiendes; solo lo escondes.
3. **El output no es la respuesta.** La respuesta es la *justificación técnica* del output. Un número sin explicación es media kata.
4. **Evita el anti-patrón de forma demostrable.** No basta con "no lo hice mal": muestra el experimento que prueba por qué el camino correcto es mejor.

El continuum del curso. Todo el material recorre una sola idea: *convertir lenguaje en vectores donde la geometría codifica el significado.* Cinco etapas, cada una resuelve una pregunta y alimenta a la siguiente:

#	Etapa	Pregunta que resuelve	Técnicas
1	Tokenización	¿Cómo partir el texto en unidades?	char · palabra · subpalabra · byte
2	Representación dispersa	¿Cómo comparar documentos?	TF · TF-IDF · coseno
3	Embeddings densos	¿Cómo meter significado en pocos números?	Word2Vec · GloVe · fastText
4	Embeddings contextuales	¿Cómo distinguir sentido según contexto?	ELMo · BERT/BETO · GPT
5	Embeddings de oración	¿Cómo comparar oraciones enteras?	SBERT · BioBERT

Las katas 01–10 recorren ese continuum en orden (de contar caracteres a mano en la Kata 01 a un buscador semántico biomédico en la Kata 10). Las katas 11–15 (Bloque E) bajan un nivel: la **arquitectura** —del RNN al Transformer— que produce los embeddings contextuales.

Bloque A — Tokenización subpalabra

La tokenización es el **primer eslabón** de todo pipeline de PLN: convierte una secuencia continua de caracteres en una lista discreta de unidades (tokens) que el modelo manipula como vectores. La decisión condiciona cuatro cosas medibles:

- **Tamaño de la matriz de embeddings** = `vocab × dim`. Cada token necesita su fila. Vocab de 50 000 con dim 768 son ~38 millones de parámetros solo en la capa de entrada.
- **Longitud media de secuencia** que ve el modelo. Importa porque la atención del Transformer cuesta **O(n²)**: duplicar la longitud cuadruplica el cómputo de atención.
- **Manejo de palabras desconocidas (OOV)** y de morfología (plurales, flexiones, derivados).
- **Transferencia entre idiomas** (cobertura unicode, scripts no latinos).

Estrategia	Unidad	Vocab típico	Ventaja	Desventaja
Carácter	1 char	~100	sin OOV, vocab mínimo	secuencias larguísimas, pierde semántica de palabra
Palabra	palabra completa	10^5–10^6	semántica directa	vocab enorme, OOV severo, mal con flexión
Subpalabra	morfema/fragmento	10^3–10^5	balance OOV ↔ longitud	requiere entrenar, no siempre lingüístico
Byte	1 byte UTF-8	256	universal multi-idioma	secuencias aún más largas que char

Todos los modelos SOTA (BERT, GPT, T5, LLaMA) usan **subpalabra**. Las katas 01 y 02 desarman ese "subpalabra" hasta el último detalle.

Kata 01 — Byte-Pair Encoding (BPE) desde cero

Objetivo. Implementar BPE entrenable sin librerías y entender, a nivel de bucle, que un tokenizador no es magia: es *conteo de pares adyacentes + fusión greedy repetida*.

Contexto y motivación

Antes de 2016 la traducción automática neuronal sufría con palabras raras: o el vocab explotaba, o todo lo no visto caía en un `<unk>` que destruía la traducción. Sennrich, Haddow & Birch (ACL 2016) tomaron prestado un algoritmo de **compresión de datos** de 1994 (Gage) y lo re-orientaron: en vez de comprimir bytes, fusionan los pares de símbolos más frecuentes de un corpus hasta llegar a un vocabulario de tamaño objetivo. El resultado es un vocabulario de subpalabras que cubre *cualquier* texto sin OOV duro.

Teoría formal

Sea `C` un corpus y `V` el vocabulario. BPE construye `V` así:

```
1. V ← conjunto de todos los caracteres que aparecen en C
   (cada palabra se escribe como secuencia de chars + marcador fin-de-palabra </w>)
2. repetir hasta |V| = tamaño_objetivo:
     a. contar la frecuencia de cada par adyacente (x, y) en el corpus actual
     b. elegir el par (a, b) más frecuente
     c. crear símbolo nuevo "ab", añadirlo a V
     d. reemplazar TODA ocurrencia de "a b" por "ab" en el corpus
3. devolver V y la lista ORDENADA de fusiones (las reglas)
```

Tres propiedades que debes poder defender:

- **Determinista.** Mismo corpus + mismo `tamaño_objetivo` → exactamente las mismas fusiones, en el mismo orden. (Con empates de frecuencia hace falta una regla de desempate estable, p. ej. orden lexicográfico, para que sea 100% reproducible.)
- **Greedy.** En cada paso elige el par localmente más frecuente y **nunca revisa** esa decisión. No garantiza la segmentación globalmente óptima — solo una buena y barata.
- **Vocab abierto.** Como el nivel base son caracteres, *cualquier* palabra futura es representable: en el peor caso, como una tira de caracteres sueltos. Por eso no hay OOV duro, solo segmentaciones más largas.

El marcador `</w>` es importante: distingue `est` final de palabra (como en `lowest</w>`) de un `est` interno. Sin él, `est` de `lowest` y de `establo` se confundirían.

Ejemplo numérico resuelto a mano

Corpus (con frecuencias): `low` ×5, `lowest` ×2, `newer` ×6, `wider` ×3.

Estado inicial (cada palabra = chars separados + `</w>`):

```
l o w </w>          : 5
l o w e s t </w>    : 2
n e w e r </w>      : 6
w i d e r </w>      : 3
```

Iteración 1 — contar pares:

```
(e,r)  = 6 (newer) + 3 (wider)         = 9   ← máximo
(l,o)  = 5 + 2                          = 7
(o,w)  = 5 + 2                          = 7
(n,e)  = 6
(w,e)  = 6
(e,w)  = 6
...
```

Par ganador: `(e, r)` con 9. Fusión #1: `e r → er`. Corpus:

```
l o w </w>          : 5
l o w e s t </w>    : 2
n e w er </w>       : 6
w i d er </w>       : 3
```

Iteración 2 — recontar:

```
(l,o)  = 7
(o,w)  = 7
(er,</w>) = 6 + 3 = 9   ← máximo
...
```

Par ganador: `(er, </w>)`. Fusión #2: `er </w> → er</w>`. Y así sucesivamente. Las primeras fusiones capturan el sufijo `–er` de comparativos, exactamente lo que un lingüista señalaría — pero el algoritmo llegó ahí **solo por conteo**, sin saber qué es un comparativo.

Implementación completa

```
from collections import Counter

def get_stats(vocab):
    """Cuenta frecuencia de pares adyacentes en el corpus tokenizado."""
    pairs = Counter()
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i + 1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    """Fusiona el par dado en todo el corpus."""
    bigram = " ".join(pair)
    replacement = "".join(pair)
    return {w.replace(bigram, replacement): f for w, f in vocab.items()}

def train_bpe(corpus_freqs, num_merges):
    # cada palabra → chars separados por espacio + marcador fin de palabra
    vocab = {" ".join(list(w)) + " </w>": f for w, f in corpus_freqs.items()}
    merges = []
    for i in range(num_merges):
        pairs = get_stats(vocab)
        if not pairs:
            break
        # desempate estable: por (-frecuencia, par) para reproducibilidad
        best = min(pairs.items(), key=lambda kv: (-kv[1], kv[0]))[0]
        merges.append(best)
        vocab = merge_vocab(best, vocab)
            print(f"merge {i+1:2d}: {best[0]!r}+{best[1]!r} -> {''.join(best)!r}   (freq=
        {pairs[best]})")
    return merges

def tokenize(word, merges):
    """Aplica las reglas aprendidas EN ORDEN a una palabra nueva."""
    symbols = list(word) + ["</w>"]
    for a, b in merges:
        i = 0
        while i < len(symbols) - 1:
            if symbols[i] == a and symbols[i + 1] == b:
                symbols[i:i + 2] = [a + b]   # fusiona en sitio
            else:
                i += 1
    return symbols

if __name__ == "__main__":
    corpus = {"low": 5, "lowest": 2, "newer": 6, "wider": 3}
    merges = train_bpe(corpus, num_merges=10)
    print("\nReglas:", merges)
    for w in ["newer", "lowering", "wildest"]:
        print(f"{w:10s} -> {tokenize(w, merges)}")
```

Salida esperada (aproximada)

```
merge  1: 'e'+'r' -> 'er'  (freq=9)
merge  2: 'er'+'</w>' -> 'er</w>'  (freq=9)
merge  3: 'l'+'o' -> 'lo'  (freq=7)
merge  4: 'lo'+'w' -> 'low'  (freq=7)
...
newer      -> ['n', 'e', 'w', 'er</w>']
lowering   -> ['low', 'e', 'r', 'i', 'n', 'g', '</w>']
wildest    -> ['w', 'i', 'l', 'd', 'e', 's', 't', '</w>']
```

Observa: `newer` se segmenta limpio porque sus piezas se entrenaron; `lowering` y `wildest` quedan muy fragmentados porque el corpus juguete no tenía `-ing` ni `-ild`. **Eso es lo esperado** y es justo la lección: la calidad de la segmentación depende del corpus.

Errores comunes / debugging

- **Olvidar `</w>`.** Sin marcador de fin no distingues sufijo de infijo; las fusiones salen raras. Si ves que `er` se fusiona dentro de `establo`, te falta el marcador.
- **No recontar pares tras cada fusión.** Las frecuencias cambian al fusionar; si reusas el conteo viejo, el algoritmo diverge.
- **Empates sin desempate estable.** Si `most_common(1)` rompe empates por orden de inserción del dict, dos corridas pueden diferir. Por eso el código usa `key=lambda kv: (-kv[1], kv[0])`.
- **Reemplazo con `str.replace` y subcadenas traposas.** Funciona aquí porque los símbolos van separados por espacio; si quitas los espacios, `replace` puede pisar fusiones parciales.

Variantes y extensiones

1. **Byte-level BPE** (GPT-2): en vez de caracteres unicode, parte de los 256 bytes UTF-8. Garantiza cobertura universal sin ningún `<unk>`, a costa de secuencias algo más largas. Modifica el `list(w)` inicial para iterar sobre `w.encode("utf-8")`.
2. **Dropout BPE** (Provilkov et al., 2020): durante el entrenamiento del *modelo* (no del tokenizador) omite aleatoriamente algunas fusiones → regulariza y mejora robustez.
3. Mide cómo cambia la longitud media de tokenización de un texto al pasar de 3 a 30 a 300 fusiones. Verás la curva char → subpalabra → casi-palabra.

Anti-patrón

> ***Creer que BPE "entiende" morfología.*** *No: solo cuenta frecuencia. Que `-er` o `-ing` salgan como piezas limpias es un efecto colateral estadístico de que esos sufijos son frecuentes, no conocimiento lingüístico. Corolario: en un corpus sesgado, BPE producirá segmentaciones sesgadas, y eso se propaga a todo el modelo aguas abajo.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Reglas reproducibles	Dos corridas dan idéntica lista de fusiones en el mismo orden
Fin de palabra	El código usa y respeta `</w>`
Tokeniza palabra nueva	`lowering` (no vista) se segmenta usando solo reglas aprendidas
Justificación	Explicas por qué `wildest` queda más fragmentado que `newer`

Reflexión

1. Si entrenas 3 fusiones vs 30, ¿cómo cambia la longitud tokenizada de `lowering`? ¿Y el tamaño del vocab?
2. ¿Por qué BPE nunca produce OOV duro, pero sí puede producir segmentaciones absurdamente largas? ¿Cuándo es eso un problema real?
3. Conecta con la Kata 07: los n-gramas de carácter de fastText resuelven OOV de otra manera. ¿En qué se parecen y en qué difieren de BPE?

Referencias

- Sennrich, Haddow, Birch (2016). *Neural Machine Translation of Rare Words with Subword Units.* ACL.
- Gage (1994). *A New Algorithm for Data Compression.* (el BPE original, de compresión.)

- Notebook base: `tokenizacion/Sentencepiece+bpe.ipynb`.

Kata 02 — Comparar 4 familias de tokenizador con SentencePiece

Objetivo. Entrenar BPE, Unigram y Char sobre el mismo corpus español (`tokenizacion/word_billion_chunks`) con SentencePiece y **cuantificar** el trade-off fundamental: vocabulario ↔ longitud de secuencia ↔ cobertura.

Contexto y motivación

La Kata 01 te dio BPE a mano. En producción nadie lo implementa: se usa **SentencePiece** (Kudo & Richardson, 2018), una librería que trata el texto como un flujo crudo de unicode (incluso los espacios, que codifica como `_`), por lo que es *language-agnostic* y reversible sin reglas de detokenización. Aquí dejas de implementar y pasas a **medir y decidir**, que es la competencia real de un arquitecto: elegir el tokenizador con datos, no por moda.

Teoría: las tres métricas que deciden

1. **Fertility (fertilidad)** = `nº de tokens / nº de palabras`. Cuánto fragmenta en promedio. ~1.0 = casi una pieza por palabra (vocab grande o idioma bien cubierto). >2 = fragmenta mucho (vocab chico o idioma mal cubierto). No existe un valor "bueno" absoluto: se interpreta contra el costo que impone.
2. **Cobertura / tasa de** `<unk>` = fracción del texto que cae en token desconocido. Con `character_coverage` alto (0.9995 para idiomas con alfabeto chico como el español; 0.9999 para chino/japonés) debería ser ≈0. Si sube, tu vocab no cubre el script.
3. **Longitud media de secuencia.** Tokens por documento. Importa por el costo **O(n²)** de la atención: si Char duplica la longitud frente a BPE, la atención cuesta ~4×. Es la métrica que traduce "fragmenta mucho" a "cuesta plata".

BPE vs Unigram: construir vs podar

- **BPE** *construye* el vocab de abajo hacia arriba: empieza con caracteres y fusiona. Es greedy y determinista. La segmentación de una palabra es única (aplica las fusiones en orden).
- **Unigram LM** (Kudo, 2018) hace lo contrario: *empieza con un vocab grande* de candidatos y los **poda** maximizando la verosimilitud de un modelo unigrama sobre el corpus. Modela cada palabra como una distribución de segmentaciones posibles y se queda con las piezas que más aportan a la verosimilitud. Suele dar segmentaciones más coherentes y permite **muestrear** segmentaciones (regularización por subword sampling).
- **Char** es el caso degenerado: vocab = alfabeto. Sin OOV, pero secuencias larguísimas.

Implementación completa

```
import sentencepiece as spm
import glob, statistics as st

CORPUS = "tokenizacion/word_billion_chunks/chunk_00.txt"   # ajusta al archivo real

configs = [
    ("bpe",     dict(model_type="bpe",     vocab_size=8000)),
    ("unigram", dict(model_type="unigram", vocab_size=8000)),
    ("char",    dict(model_type="char",    vocab_size=512)),
]

for name, cfg in configs:
    spm.SentencePieceTrainer.train(
        input=CORPUS,
        model_prefix=f"sp_{name}",
        character_coverage=0.9995,    # español: alfabeto chico
        **cfg,
    )

# texto de evaluación (NO usado en entrenamiento idealmente)
eval_text = [
    "Los tokenizadores subpalabra reducen el problema de palabras desconocidas.",
    "La paralelización electroencefalográfica resultó inesperadamente eficiente.",
    "¡Qué desafortunadísimo contratiempo, che!",
]

print(f"{'modelo':8s} {'fertility':>10s} {'len_media':>10s} {'unk%':>6s}")
for name, _ in configs:
    sp = spm.SentencePieceProcessor(model_file=f"sp_{name}.model")
    n_words = n_tok = n_unk = 0
    lengths = []
    for line in eval_text:
        ids = sp.encode(line, out_type=int)
        pieces = sp.encode(line, out_type=str)
        n_words += len(line.split())
        n_tok   += len(pieces)
        n_unk   += sum(1 for i in ids if i == sp.unk_id())
        lengths.append(len(pieces))
         print(f"{name:8s}  {n_tok/n_words:10.2f}  {st.mean(lengths):10.1f}  {100*n_unk/n_-
        tok:6.2f}")

# inspección cualitativa: cómo parte una palabra larga
for name, _ in configs:
    sp = spm.SentencePieceProcessor(model_file=f"sp_{name}.model")
    print(name, "->", sp.encode("electroencefalográfica", out_type=str))
```

Salida esperada (forma; los números dependen del corpus)

```
modelo    fertility  len_media   unk%
bpe            1.65       13.0   0.00
unigram        1.52       12.0   0.00
char           6.80       54.0   0.00

bpe     -> ['_electro', 'ence', 'fal', 'ográ', 'fica']
unigram -> ['_electro', 'encefalo', 'gráfica']
char    -> ['_','e','l','e','c','t','r','o', ... ]
```

Lectura: Char nunca tiene `<unk>` pero su longitud es ~4× la de subpalabra → atención ~16× más cara. Unigram suele fragmentar un poco menos que BPE y con cortes más "morfológicos" (`encefalo` , `gráfica`).

Errores comunes / debugging

- **Evaluar sobre el mismo texto de entrenamiento.** Infla cobertura y baja fertility de forma engañosa. Usa un texto held-out.
- `character_coverage=1.0` **en idiomas con muchos caracteres raros.** Mete símbolos basura al vocab. Para español 0.9995 está bien; para CJK usa 0.9999.
- **Comparar vocab_size distintos entre modelos.** Si BPE tiene 8k y Unigram 32k, la comparación de fertility no es justa. Fija el mismo `vocab_size` .
- **Confundir "menos tokens" con "mejor".** Menos tokens = vocab más grande = matriz de embeddings más cara y peor generalización a palabras raras. Hay óptimo, no máximo.

Variantes y extensiones

1. Barrido de `vocab_size` ∈ {2k, 8k, 16k, 32k} y grafica fertility vs vocab. Verás rendimientos decrecientes: pasado cierto punto, duplicar el vocab casi no baja la fertility pero duplica los parámetros de embedding.
2. Activa `enable_sampling=True` en Unigram y tokeniza la misma frase 5 veces: obtienes segmentaciones distintas (subword regularization). Útil como data augmentation.
3. Mide fertility en español vs inglés con el *mismo* tokenizador multilingüe. Los idiomas flexivos (español) suelen fragmentar más → "impuesto de tokenización" por idioma.

Anti-patrón

> ***Elegir el vocab más grande "porque fragmenta menos".*** *Vocab grande = matriz de embeddings* `vocab × dim` *enorme (memoria + parámetros) y peor generalización: piezas raras apenas se ven en entrenamiento y aprenden vectores pobres. El objetivo es el* ***punto de equilibrio*** *entre longitud de secuencia y tamaño de vocab para tu presupuesto de cómputo, no minimizar la fertility a cualquier costo.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Tres modelos entrenados	`sp_bpe`, `sp_unigram`, `sp_char` con mismo `vocab_size` cuando aplica
Tabla de métricas	fertility, longitud media y `unk%` por modelo, sobre texto held-out
Decisión justificada	Eliges uno para un Transformer en español y argumentas con la tabla y el costo $O(n^2)$
Inspección cualitativa	Muestras cómo cada modelo parte una palabra larga y comentas

Reflexión

1. Si Char duplica la longitud frente a BPE, ¿cuánto sube el cómputo de **atención**? ¿Y el de las capas feed-forward (que son O(n))?
2. ¿Por qué el español tiende a fragmentar más que el inglés con el mismo vocab?
3. ¿En qué caso elegirías Char a pesar del costo? (pista: idiomas sin espacios, datos muy ruidosos, o cuando el OOV cero es crítico.)

Referencias

- Kudo (2018). *Subword Regularization: Improving NMT Models with Multiple Subword Candidates.* (Unigram LM.)
- Kudo & Richardson (2018). *SentencePiece: A simple and language independent subword tokenizer.* EMNLP demos.

- Notebooks base: `tokenizacion/act4-tokenizadores.ipynb`, `tokenizacion/tokenizadores_transformers.ipynb`.

Bloque B — Semántica vectorial clásica (representación dispersa)

Ya sabes partir el texto en unidades. Ahora la pregunta cambia: **¿cómo comparar dos documentos?** La respuesta clásica (anterior a los embeddings densos, y aún hoy un baseline durísimo) es el **Vector Space Model**: cada documento es un punto en un espacio de tantas dimensiones como términos tenga el vocabulario. La similitud entre documentos = cercanía geométrica entre sus vectores. Estas representaciones son **dispersas**: un documento toca unos cientos de los $\sim 10^4$–10^5 términos del vocab, así que su vector es casi todo ceros.

Kata 03 — VSM + TF-IDF + similitud coseno sobre Shakespeare

Objetivo. Reproducir el caso canónico de Jurafsky & Martin (*Speech and Language Processing*, cap. 6): representar cuatro obras de Shakespeare como vectores TF-IDF y rankearlas por similitud coseno, entendiendo cada término de la fórmula.

Teoría formal

Term Frequency (TF). Cuántas veces aparece el término `t` en el documento `d`. La cuenta cruda `count(t,d)` es mala señal: que una palabra aparezca 100 veces no la hace 100× más importante que una que aparece 1 vez. Por eso se aplana con log:

```
tf(t,d) = 1 + log10(count(t,d))   si count > 0,   y 0 en otro caso
```

Document Frequency e Inverse Document Frequency (IDF). `df(t)` = en cuántos documentos (de `N`) aparece `t`. Términos que salen en *todos* los documentos ("the", "y", "de") no discriminan nada. IDF los castiga:

```
idf(t) = log10( N / df(t) )
```

Un término en los `N` documentos: `idf = log(1) = 0` → se anula. Un término en 1 solo documento: `idf` máximo. **TF-IDF** = `tf(t,d) × idf(t)`: alto para términos frecuentes en *este* documento pero raros en el corpus → justo los que caracterizan al documento.

> ***Nota sobre scikit-learn.*** *Su `TfidfVectorizer` usa por defecto una variante suavizada: `idf(t) = ln((1+N)/(1+df(t))) + 1` (el `+1` evita que un término ubicuo tenga peso 0, y el suavizado evita división por cero), y luego **normaliza L2** cada vector de documento. Por eso los números de sklearn no coinciden con la fórmula de Jurafsky, pero el ranking es consistente.*

Similitud coseno. Mide el **ángulo** entre dos vectores, no la distancia:

```
cos(a, b) = (a · b) / (‖a‖₂ · ‖b‖₂) = Σᵢ aᵢbᵢ / (√Σaᵢ² · √Σbᵢ²)
```

Con vectores no negativos (como TF-IDF) el rango es [0, 1]: 1 = misma dirección (máxima similitud), 0 = ortogonales (sin términos en común). La clave: el coseno **ignora la magnitud**, por lo que un documento largo y uno corto sobre el mismo tema salen similares. La distancia euclídea no tiene esa propiedad — y por eso aquí se usa coseno.

Ejemplo numérico resuelto a mano

Tres "documentos" minúsculos, vocab = {gato, perro, casa}:

```
d1 = "gato gato perro"   → conteo: gato=2, perro=1, casa=0
d2 = "gato perro perro"  → conteo: gato=1, perro=2, casa=0
d3 = "casa casa casa"    → conteo: gato=0, perro=0, casa=3
```

IDF (N=3, sin suavizar): `gato` en d1,d2 → df=2 → idf=log10(3/2)=0.176. `perro` igual=0.176. `casa` en d3 → df=1 → idf=log10(3)=0.477.

Vector TF-IDF de d1 (usando tf cruda para simplificar): `[2×0.176, 1×0.176, 0] = [0.352, 0.176, 0]`. d3: `[0, 0, 3×0.477] = [0, 0, 1.431]`.

`cos(d1, d3)` = 0 (no comparten términos). `cos(d1, d2)` > 0 (comparten gato y perro). A mano confirmas que d1 y d2 son los más parecidos y d3 el outlier — sin tocar un computador.

Implementación completa

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

nombres = ["hamlet", "macbeth", "julius_caesar", "tempest"]
docs = [open(f"obras/{n}.txt", encoding="utf-8").read() for n in nombres]

vec = TfidfVectorizer(stop_words="english", min_df=1, sublinear_tf=True)
X = vec.fit_transform(docs)          # matriz dispersa (4 × |vocab|)
sim = cosine_similarity(X)           # 4 × 4, diagonal = 1

print("   " + "  ".join(f"{n[:6]:>6s}" for n in nombres))
for i, n in enumerate(nombres):
    print(f"{n[:6]:>6s} " + "  ".join(f"{sim[i,j]:6.2f}" for j in range(len(nombres))))

# par más similar (ignorando la diagonal)
np.fill_diagonal(sim, 0)
i, j = np.unravel_index(sim.argmax(), sim.shape)
print(f"\nMás similares: {nombres[i]} ↔ {nombres[j]} (cos={sim[i,j]:.3f})")

# términos que más contribuyen a ese par
terms = np.array(vec.get_feature_names_out())
contrib = X[i].toarray()[0] * X[j].toarray()[0]      # producto término a término
top = contrib.argsort()[::-1][:10]
print("Términos compartidos de mayor peso:", terms[top].tolist())
```

Salida esperada (forma)

```
       hamlet  macbet  julius  tempes
hamlet   1.00    0.42    0.38    0.31
macbet   0.42    1.00    0.40    0.29
julius   0.38    0.40    1.00    0.27
tempes   0.31    0.29    0.27    1.00

Más similares: hamlet ↔ macbeth (cos=0.420)
Términos compartidos de mayor peso: ['lord','king','death','blood', ...]
```

La justificación NO es "0.42 es el número más alto"; es "Hamlet y Macbeth comparten léxico de realeza, muerte y traición (`lord` , `king` , `death` , `blood`), que TF-IDF resalta porque son frecuentes en ambas pero no ubicuos en todo el corpus".

Errores comunes / debugging

- **No quitar stopwords.** Sin `stop_words` , los términos de mayor frecuencia dominan la cuenta cruda; aunque IDF los baja, ensucian. (En español usa una lista propia o `min_df` .)
- **Usar conteo crudo sin `sublinear_tf` .** Una palabra repetida 500 veces infla el vector. `sublinear_tf=True` aplica el `1+log(tf)` .
- **Comparar con distancia euclídea.** Penaliza por longitud de documento → Hamlet (largo) parece lejano de todo. Es el error que motiva el coseno (ver Kata 04).
- **Leer la diagonal como hallazgo.** `sim[i,i]=1` siempre; recuerda anularla antes del `argmax` .

Variantes y extensiones

1. Repite con **bigramas** (`ngram_range=(1,2)`). Captura colocaciones ("my lord") y suele subir la nitidez del ranking.
2. Construye la matriz **término-documento** y aplica TF-IDF sobre términos para encontrar, por cada obra, sus 10 palabras más características (mayor TF-IDF). Es "keyword extraction" gratis.
3. Cambia el corpus a tus propios textos del taller y verifica que el par temáticamente más cercano sale primero.

Anti-patrón

> ***Bag-of-Words con conteo crudo + distancia euclídea.*** *Sin IDF, las palabras comunes dominan; con euclídea, la magnitud (longitud del documento) domina. El resultado mide "tamaño parecido", no "tema parecido". TF-IDF + coseno existe precisamente para neutralizar ambos sesgos: IDF mata lo ubicuo, el coseno mata la magnitud.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Matriz 4×4	Impresa, simétrica, diagonal 1
Par más cercano	Identificado correctamente excluyendo la diagonal
Justificación léxica	Listas los términos compartidos de mayor peso y explicas el sentido
Comprensión de IDF	Explicas por qué una palabra ubicua tiene peso ≈0

Reflexión

1. ¿Por qué dos obras del *mismo autor* pueden salir poco similares en TF-IDF aunque "suenen" parecidas? (pista: TF-IDF ve léxico, no estilo ni sintaxis.)
2. Si añades una quinta obra que es una secuela de Hamlet, ¿cómo cambian las IDF de los términos de realeza?
3. ¿Qué información se pierde irremediablemente al pasar a Bag-of-Words? (orden, negación, correferencia...) — esto motiva los embeddings contextuales de la Kata 09.

Referencias

- Jurafsky & Martin, *Speech and Language Processing*, cap. 6 (Vector Semantics & TF-IDF).
- Notebook base: `semantica/act6-semantica.ipynb`.

Kata 04 — Normalización L2: por qué importa (y cuándo no)

Objetivo. Demostrar **empíricamente** que (a) tras normalizar L2 el producto punto *es* el coseno, y (b) sin normalizar, el ranking de similitud se distorsiona por la longitud del documento.

Teoría formal

La norma L2 (euclídea) de un vector: $\|v\|_2 = \sqrt{(\Sigma_i\ v_i^2)}$. Normalizar L2:

$$\hat{v} = v\ /\ \|v\|_2$$

Tras la normalización todo vector tiene norma 1 → vive en la **esfera unidad**. Y entonces:

$$\hat{v} \cdot \hat{w} = (v/\|v\|)\cdot(w/\|w\|) = (v\cdot w)/(\|v\|\|w\|) = \cos(v, w)$$

Es decir, **en vectores L2-normalizados el producto punto coincide exactamente con el coseno.** Esto tiene una consecuencia práctica enorme: en un buscador semántico precalculas todos los embeddings normalizados y luego el ranking por coseno es un simple producto matriz–vector (rapidísimo, y lo que hace FAISS con índices de producto interno).

`TfidfVectorizer` de sklearn normaliza L2 por defecto (`norm='l2'`). Esta kata apaga la normalización para *ver* el efecto.

Ejemplo numérico resuelto a mano

Tres documentos sobre vocab {gato, negro, perro, grande, corre, parque}:

```
d1 = "gato negro"                (corto)
d2 = "gato negro gato negro gato"(corto, repetido — mismo tema que d1)
d3 = "perro grande corre parque"(otro tema)
```

Con conteo crudo (sin normalizar), el producto punto `d1·d2` crece con las repeticiones de d2, y `d2·d2` (su "auto-similitud") es enorme → d2 parece similar a *todo* solo por tener más cuenta. Al normalizar L2, d1 y d2 tienen casi `cos=1` (mismo tema, distinta longitud) y ambos `cos≈0` con d3. La normalización revela que d1 y d2 son "el mismo documento a distinta escala".

Implementación completa

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "gato negro",
    "gato negro gato negro gato",
    "perro grande corre por el parque rápido",
]

for norm in [None, "l2"]:
    X = TfidfVectorizer(norm=norm).fit_transform(docs).toarray()
    dot = X @ X.T                       # producto punto crudo
    print(f"\n=== norm={norm} ===")
    print(np.round(dot, 3))
    print("‖dᵢ‖:", np.round(np.linalg.norm(X, axis=1), 3))
```

Salida esperada (forma)

```
=== norm=None ===
[[0.78  1.31  0.  ]
 [1.31  3.4   0.  ]      ← d2·d2 = 3.4 » resto: magnitud domina
 [0.    0.    5.2 ]]
‖dᵢ‖: [0.88 1.84 2.28]   ← normas MUY distintas

=== norm=l2 ===
[[1.    0.97  0.  ]
 [0.97  1.    0.  ]      ← d1 y d2: casi idénticos (mismo tema)
 [0.    0.    1.  ]]
‖dᵢ‖: [1. 1. 1.]         ← todos en la esfera unidad
```

Sin normalizar, `d2·d2=3.4` y `d1·d2=1.31` no son comparables entre filas porque cada documento tiene una escala distinta. Normalizado, queda claro que d1≈d2 y ambos ⊥ d3.

Errores comunes / debugging

- **Asumir que sklearn no normaliza.** Por defecto SÍ (`norm='l2'`). Para ver el efecto crudo hay que pasar `norm=None` explícitamente.
- **Normalizar dos veces.** Si tu vectorizador ya normalizó, volver a normalizar es inocuo pero señala que no entiendes el pipeline.
- **Usar producto punto crudo como "similitud" sin normalizar.** Es el error que esta kata expone: parece que funciona en documentos de longitud parecida y revienta cuando varían.

Variantes y extensiones

1. Compara L2 con normalización L1 (`v/Σ|vᵢ|`). L1 da una distribución de probabilidad; útil para divergencias (KL, Jensen-Shannon) en vez de coseno.
2. Demuestra numéricamente la identidad `v̂·ŵ = cos(v,w)` calculando ambos lados y verificando que difieren en <1e-9.

3. Mide el speedup: ranking por `X_norm @ q_norm` vs llamar `cosine_similarity` repetidamente.

Anti-patrón

> "***Más palabras compartidas = más similar***". *Falso sin normalizar: un documento largo comparte muchas palabras con todo por pura masa. La normalización L2 convierte "cuánto comparten" en "qué fracción de su dirección comparten", que es lo que de verdad quieres.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Dos matrices	`norm=None` y `norm='l2'` impresas y contrastadas
Cambio de ranking	Muestras un par cuyo orden de similitud cambia entre ambas
Identidad demostrada	Verificas numéricamente `û·ŵ = cos(v,w)`
Decisión	Explicas cuál es el ranking correcto y por qué

Reflexión

1. ¿En qué caso querrías **no** normalizar? (pista: cuando la magnitud ES señal — p. ej. la confianza o el peso/importancia de un documento debe influir.)
2. Si todos los vectores están L2-normalizados, ¿qué relación hay entre distancia euclídea y coseno entre ellos? (pista: `‖â−b̂‖² = 2 − 2·cos`.)
3. ¿Por qué los índices de búsqueda vectorial (FAISS) prefieren producto interno sobre vectores normalizados en vez de calcular coseno explícito?

Referencias

- Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, cap. 6.
- Jurafsky & Martin, cap. 6 (cosine & normalization).

Kata 05 — Reducción de dimensión: PCA vs t-SNE

Objetivo. Proyectar vectores TF-IDF (o embeddings) a 2D para visualizar, y entender —de forma demostrable— **qué preserva y qué distorsiona** cada método, para no sacar conclusiones inválidas. Tienes imágenes de referencia en `semantica/pca_*.png` y `semantica/tsne_*.png`.

Teoría formal

PCA (Principal Component Analysis). Método **lineal**. Encuentra las direcciones ortogonales de **máxima varianza** de los datos (los autovectores de la matriz de covarianza, ordenados por autovalor). Proyectar a 2D = quedarse con las 2 direcciones de mayor varianza.

```
1. centrar: X̃ = X − media
2. covarianza: C = (1/n) X̃ᵀX̃
3. autovectores/autovalores de C: C vᵢ = λᵢ vᵢ
4. proyectar sobre los 2 vᵢ de mayor λᵢ
```

Propiedades: **determinista**, ejes interpretables, preserva **estructura global** y distancias grandes. La fracción de varianza explicada por las 2 primeras componentes (`explained_variance_ratio_`) te dice cuánto estás perdiendo.

t-SNE (t-distributed Stochastic Neighbor Embedding). Método **no lineal**. No preserva distancias; preserva **vecindad local**. Convierte distancias en probabilidades de vecindad (gaussiana en alta dimensión, controlada por la **perplexity** ≈ "nº efectivo de vecinos"), hace lo mismo en 2D con una **t-Student** (colas pesadas → evita el "crowding"), y minimiza la divergencia **KL** entre ambas distribuciones por descenso de gradiente.

Consecuencias críticas que debes poder enunciar:

- **Es estocástico** → fija `random_state` o cada corrida cambia el layout.
- **El tamaño de los clusters no significa nada** (t-SNE los infla/encoge).
- **La distancia ENTRE clusters no significa nada** — dos clusters muy separados no son "más distintos" que dos cercanos. Solo es fiable la *pertenencia* a un cluster.
- La `perplexity` cambia el resultado: baja → muchos micro-clusters; alta → todo se funde.

Regla de uso: PCA para diagnóstico rápido y estructura global; t-SNE (o UMAP) para *ver* agrupamientos, sabiendo que solo la vecindad local es interpretable.

Implementación completa

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

X = tfidf_matrix.toarray()          # de la Kata 03, o cualquier matriz de vectores
labels = nombres                    # etiquetas para colorear/anotar

# --- PCA ---
pca = PCA(n_components=2, random_state=0)
P = pca.fit_transform(X)
var = pca.explained_variance_ratio_
print(f"Varianza explicada por PC1+PC2: {var.sum():.1%}  ({var[0]:.1%} + {var[1]:.1%})")

# --- t-SNE (perplexity < n_muestras) ---
perp = min(5, len(X) - 1)
T = TSNE(n_components=2, perplexity=perp, random_state=0, init="pca").fit_transform(X)

fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
for axis, data, title in [(ax[0], P, f"PCA ({var.sum():.0%} var)"),
                          (ax[1], T, f"t-SNE (perp={perp})")]:
    axis.scatter(data[:, 0], data[:, 1])
    for k, name in enumerate(labels):
        axis.annotate(name, (data[k, 0], data[k, 1]))
    axis.set_title(title)
plt.tight_layout()
plt.savefig("kata05_proj.png", dpi=120)
print("guardado kata05_proj.png")
```

Salida esperada (forma)

```
Varianza explicada por PC1+PC2: 58.0%  (34.0% + 24.0%)
guardado kata05_proj.png
```

Si la varianza explicada es baja (p. ej. 15%), el dibujo PCA 2D es una sombra muy parcial de un espacio intrínsecamente alto-dimensional — TF-IDF típicamente lo es. Eso *en sí* es un hallazgo: justifica por qué luego querrás embeddings densos de baja dimensión (Bloque C).

Errores comunes / debugging

- `perplexity ≥ n_muestras` → t-SNE falla. Debe ser bastante menor que el nº de puntos.
- **No fijar** `random_state` → "el cluster se movió" entre corridas; no es señal, es ruido.
- **Leer distancias entre clusters de t-SNE** como magnitudes reales. Inválido.
- **No estandarizar antes de PCA** cuando las features tienen escalas dispares. (Con TF-IDF L2-normalizado suele estar bien; con features crudas, estandariza.)

Variantes y extensiones

1. Sustituye t-SNE por **UMAP**: preserva algo más de estructura global, escala mejor y es más estable. Compara los tres (PCA, t-SNE, UMAP) sobre el mismo dato.
2. Grafica la **curva de varianza explicada acumulada** de PCA (10–50 componentes) para ver cuántas dimensiones hacen falta para capturar el 90%.
3. Repite con embeddings densos (Kata 06/10) en vez de TF-IDF: verás clusters mucho más limpios porque el espacio ya está "ordenado por semántica".

Anti-patrón

> ***Interpretar el layout de t-SNE como un mapa métrico.*** *"Estos dos clusters están lejos, luego son muy distintos" — falso. Y publicar un t-SNE sin* `random_state` *fijo, de modo que nadie (ni tú) puede reproducir la figura. t-SNE responde qué agrupa con qué, no cuánto dista de qué.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Dos paneles	PCA y t-SNE lado a lado, guardados
Varianza explicada	Reportada para PCA; comentas qué implica su valor
Conclusión válida	Enuncias 1 conclusión correcta de cada método
Conclusión inválida	Enuncias 1 conclusión que NO debes sacar de t-SNE y por qué

Reflexión

1. Si PCA 2D explica solo 15% de la varianza, ¿qué dice eso sobre la dimensionalidad real de TF-IDF? ¿Cómo lo "arreglan" los embeddings densos?
2. ¿Por qué t-SNE usa una t-Student (colas pesadas) en baja dimensión en vez de otra gaussiana? (pista: el problema del "crowding".)
3. Dadas dos figuras t-SNE con perplexity 5 y 50 del mismo dato, ¿cuál creerías y por qué ninguna es "la verdadera"?

Referencias

- van der Maaten & Hinton (2008). *Visualizing Data using t-SNE.* JMLR.
- Wattenberg et al. (2016). *How to Use t-SNE Effectively* (distill.pub) — lectura obligada.
- McInnes et al. (2018). *UMAP.*

- Imágenes base: `semantica/pca_raw.png`, `pca_tfidf.png`, `tsne_raw.png`, `tsne_tfidf.png`.

Bloque C — Embeddings densos

TF-IDF tiene dos problemas estructurales: es **disperso y enorme** (10^4–10^5 dimensiones, casi todas cero) y **no captura significado** — `gato` y `felino` son tan ortogonales como `gato` y `termodinámica`, porque cada término es su propia dimensión. Los embeddings densos resuelven ambos: cada palabra es un vector **denso y corto** (100–300 dims) aprendido de modo que palabras de contexto parecido queden cerca. La base teórica es la **hipótesis distribucional** (Firth, 1957): "*You shall know a word by the company it keeps.*" El significado de una palabra está en las palabras que la rodean.

Kata 06 — Word2Vec Skip-gram con Negative Sampling, desde cero

Objetivo. Entrenar embeddings densos a mano (numpy) para entender que "significado = contexto" se vuelve un problema de clasificación binaria entrenable por gradiente. Base: `semantica/Word2vec_scratch.ipynb`.

Teoría formal

Skip-gram (Mikolov et al., 2013): dada la palabra **central** `c`, predecir sus palabras de **contexto** `o` (las de una ventana ±m). (CBOW hace lo inverso: del contexto predice el centro. Skip-gram funciona mejor con palabras raras.)

El softmax completo es inviable: `P(o|c) = exp(uₒ·v_c) / Σ_w exp(u_w·v_c)` exige sumar sobre **todo el vocabulario** en cada paso (millones de términos). **Negative Sampling** (Mikolov et al., 2013, "*Distributed Representations...*") lo reemplaza por una tarea barata: distinguir los pares **reales** (centro, contexto) de pares **falsos** (centro, palabra aleatoria).

Para cada par positivo `(c, o)` se muestrean `k` negativos `wᵢ` (típico k=5–20 en corpus chicos) y se minimiza:

```
J = − log σ(uₒ · v_c)  −  Σᵢ₌₁ᵏ log σ(−u_wᵢ · v_c)
```

donde `σ(x)=1/(1+e^-x)`. Interpretación: empuja `v_c` y `uₒ` a alinearse (producto punto alto → σ→1), y empuja `v_c` lejos de los negativos (producto negativo → σ(−·)→1).

Gradientes (los que implementas a mano):

```
∂J/∂v_c = (σ(uₒ·v_c) − 1)·uₒ  +  Σᵢ σ(u_wᵢ·v_c)·u_wᵢ
∂J/∂uₒ  = (σ(uₒ·v_c) − 1)·v_c
∂J/∂u_wᵢ= σ(u_wᵢ·v_c)·v_c
```

Hay **dos** matrices de embeddings: `V` (centro, "input") y `U` (contexto, "output"). Al final se suele usar `V` (o `V+U`) como los embeddings de palabra.

Dos trucos que cambian mucho la calidad:

- **Distribución de negativos** `P(w) ∝ freq(w)^0.75`. El exponente 0.75 aplana la distribución: muestrea palabras raras un poco más de lo que su frecuencia diría, y palabras ultra-frecuentes un poco menos. Empíricamente óptimo en el paper original.
- **Subsampling de frecuentes.** Palabras como "de", "la" se descartan con probabilidad creciente en su frecuencia → el modelo no malgasta capacidad en ellas.

Ejemplo numérico (intuición a mano)

Corpus: `"el gato come pescado"`, ventana ±1. Pares positivos: (gato, el), (gato, come), (come, gato), (come, pescado)... Para el par positivo (gato, come) muestreas un negativo, p. ej. "el". El modelo aprende: `v_gato·u_come` debe subir, `v_gato·u_el` debe bajar. Tras miles de oraciones, palabras que comparten contextos (`come` , `devora`) acaban con `u` parecidos, y sus centros con `v` parecidos → coseno alto.

Implementación completa (núcleo)

```
import numpy as np
from collections import Counter

def build_vocab(tokens, min_count=1):
    freqs = Counter(tokens)
    vocab = {w: i for i, (w, c) in enumerate(freqs.items()) if c >= min_count}
    inv = {i: w for w, i in vocab.items()}
    return vocab, inv, freqs

def train_skipgram(tokens, dim=50, window=2, k=5, epochs=5, lr=0.05, seed=0):
    rng = np.random.default_rng(seed)
    vocab, inv, freqs = build_vocab(tokens)
    Vn = len(vocab)
    ids = [vocab[t] for t in tokens if t in vocab]

    # distribución de negativos ∝ freq^0.75
    f = np.array([freqs[inv[i]] for i in range(Vn)], dtype=float) ** 0.75
    neg_p = f / f.sum()

    V = (rng.standard_normal((Vn, dim)) * 0.01)   # centro
    U = (rng.standard_normal((Vn, dim)) * 0.01)   # contexto
    sigmoid = lambda x: 1.0 / (1.0 + np.exp(-np.clip(x, -30, 30)))

    for ep in range(epochs):
        loss = 0.0
        for pos in range(len(ids)):
            c = ids[pos]
            lo, hi = max(0, pos - window), min(len(ids), pos + window + 1)
            for ctx in range(lo, hi):
                if ctx == pos:
                    continue
                o = ids[ctx]
                negs = rng.choice(Vn, size=k, p=neg_p)
                # forward + grad
                score_o = sigmoid(U[o] @ V[c])
                g_o = (score_o - 1.0)
                grad_vc = g_o * U[o]
                U[o] -= lr * g_o * V[c]
                for w in negs:
                    s = sigmoid(U[w] @ V[c])
                    grad_vc += s * U[w]
                    U[w] -= lr * s * V[c]
                V[c] -= lr * grad_vc
                loss += -np.log(score_o + 1e-9)
        print(f"epoch {ep+1}: loss/pair ≈ {loss/len(ids):.4f}")
    return V, vocab, inv

def nearest(word, V, vocab, inv, k=5):
    v = V[vocab[word]]
    sims = (V @ v) / (np.linalg.norm(V, axis=1) * np.linalg.norm(v) + 1e-9)
    order = sims.argsort()[::-1]
    return [(inv[i], round(float(sims[i]), 3)) for i in order if i != vocab[word]][:k]

if __name__ == "__main__":
    text = open("semantica/corpus_clase.txt", encoding="utf-8").read().lower().split()
    V, vocab, inv = train_skipgram(text, dim=50, epochs=5)
    for w in ["rey", "ciudad", "amor"]:
        if w in vocab:
            print(w, "->", nearest(w, V, vocab, inv))
```

Salida esperada (forma)

```
epoch 1: loss/pair ≈ 0.81
...
epoch 5: loss/pair ≈ 0.34
rey   -> [('reina', 0.71), ('príncipe', 0.66), ('trono', 0.61), ...]
```

En corpus pequeños los vecinos no serán perfectos — la lección es ver la *loss bajando* y al menos un puñado de vecinos plausibles. La analogía `rey − hombre + mujer ≈ reina` solo emerge con corpus grandes; con el corpus de clase, conténtate con vecinos coherentes.

Errores comunes / debugging

- **Overflow en `exp`.** `σ` sin `np.clip` explota con productos grandes. El `clip(−30,30)` lo evita.
- **No subsamplear frecuentes.** "de/la/el" dominan los pares positivos y los vectores colapsan. Añade subsampling si los vecinos salen todos stopwords.
- **Usar `U` en vez de `V`** (o no probar `V+U`). Suele preferirse `V`; experimenta.
- **`lr` muy alto** → loss diverge (sube). Bájalo a 0.01.
- **Vocab con `min_count=1` en corpus ruidoso** → muchas palabras vistas 1 vez, vectores basura. Sube `min_count`.

Variantes y extensiones

1. Implementa **CBOW** (promedia los contextos para predecir el centro) y compara vecinos.
2. Cambia el exponente de la distribución de negativos a 1.0 y a 0.0 (uniforme) y observa el deterioro: justifica empíricamente el 0.75.
3. Entrena con `gensim.models.Word2Vec` sobre el corpus completo y verifica que las analogías (`most_similar(positive=['rey','mujer'], negative=['hombre'])`) ahora sí funcionan.

Anti-patrón

> ***Softmax completo sobre todo el vocab** (lentísimo, inviable en corpus reales) **o no subsamplear las palabras ultra-frecuentes** (que dominan el gradiente y producen vectores donde todo vecino es una stopword). Negative sampling + subsampling no son detalles: son lo que hace a Word2Vec entrenable y útil.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Loss decreciente	La pérdida por par baja a lo largo de las épocas
Negative sampling	Implementado con distribución `freq^0.75`, no softmax completo
Vecinos plausibles	Al menos una palabra tiene k-vecinos semánticamente coherentes
Gradientes correctos	El código refleja las fórmulas de `∂J/∂v_c`, `∂J/∂u_o`, `∂J/∂u_w`

Reflexión

1. ¿Por qué `freq^0.75` y no `freq^1.0`? ¿Qué le pasa a las palabras raras con cada exponente?
2. ¿Por qué Skip-gram supera a CBOW en palabras raras, y CBOW es más rápido?
3. ¿Qué relación hay entre "k negativos" y el ruido/velocidad del entrenamiento?
4. Word2Vec da **un** vector por palabra. ¿Qué pasa con `banco` (dinero/asiento)? — abre la Kata 09.

Referencias

- Mikolov, Chen, Corrado, Dean (2013). *Efficient Estimation of Word Representations in Vector Space.* (CBOW & Skip-gram.)
- Mikolov et al. (2013). *Distributed Representations of Words and Phrases and their Compositionality.* (negative sampling, subsampling, el 0.75.)

- Notebook base: `semantica/Word2vec_scratch.ipynb`, `semantica/word2vec_clase_-text.ipynb`.

Kata 07 — GloVe vs fastText: el problema OOV y la morfología

Objetivo. Contrastar embeddings de **palabra completa** (GloVe/Word2Vec) contra fastText (**subpalabra**) en una palabra fuera de vocabulario, y entender por qué fastText brilla en español (idioma morfológicamente rico).

Teoría formal

GloVe (Pennington, Socher, Manning, 2014). En vez de la ventana deslizante local de Word2Vec, GloVe parte de la **matriz de co-ocurrencia global** `X` (cuántas veces `j` aparece en el contexto de `i` en todo el corpus) y aprende vectores tales que su producto punto aproxima el **logaritmo** de la co-ocurrencia:

$$J = \sum_{ij} f(X_{ij}) \cdot \left(w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

`f` es una función de ponderación que evita que las co-ocurrencias muy frecuentes dominen. GloVe combina lo mejor de ambos mundos (estadística global + estructura lineal). Pero, como Word2Vec, da **un vector por palabra completa** → una palabra no vista en entrenamiento **no tiene vector** (OOV duro).

fastText (Bojanowski, Grave, Joulin, Mikolov, 2017). Extiende Skip-gram: cada palabra se representa como la **suma de los vectores de sus n-gramas de carácter** (típico n=3 a 6) más el de la palabra entera. Ejemplo, `donde` con n=3 y marcadores `<`, `>`:

```
<do, don, ond, nde, de>   +   <donde>
```

Consecuencia decisiva: una palabra **nunca vista** (`tokenizando`, un neologismo, un typo `embeddings`) se reconstruye sumando los n-gramas que **sí** se entrenaron → **tiene vector útil**. Por eso fastText gana en:

- **Morfología rica** (español: `niño/niña/niños/niñez` comparten n-gramas → vectores relacionados "gratis").
- **OOV y typos** (redes sociales, dominios técnicos).
- **Idiomas aglutinantes** (turco, finés).

Implementación completa

```
import fasttext, fasttext.util

# Opción A: modelo pre-entrenado español (descarga ~7GB la primera vez)
# fasttext.util.download_model('es', if_exists='ignore')
ft = fasttext.load_model("cc.es.300.bin")

pruebas = [
    "tokenizando",       # forma flexiva poco común
    "embedings",         # TYPO deliberado de "embeddings"
    "antidisturbios",    # palabra compuesta
    "xyzqwk",            # basura: hasta esto recibe un vector (de n-gramas)
]
for w in pruebas:
    print(f"\n{w}:")
    for vecino, score in ft.get_nearest_neighbors(w, k=5):
        print(f"   {score:.3f}  {vecino}")

# Contraste con palabra completa: Word2Vec/GloVe daría KeyError aquí
print("\nVector de 'embedings' (typo) existe:", ft.get_word_vector("embedings").shape)
```

Salida esperada (forma)

```
tokenizando:
   0.78  tokenizar
   0.74  tokenización
   ...
embedings:
   0.81  embeddings        ← el typo cae cerca de la forma correcta
   0.69  incrustaciones
   ...
Vector de 'embedings' (typo) existe: (300,)
```

Un modelo de palabra completa (GloVe, Word2Vec sin subword) lanzaría `KeyError: 'embedings'` o devolvería el vector de `<unk>`. fastText reconstruye un vector razonable desde los n-gramas compartidos con `embeddings`.

Errores comunes / debugging

- **Cargar el `.vec` en vez del `.bin`.** Solo el `.bin` contiene los n-gramas de subpalabra; el `.vec` son vectores de palabra completa → pierdes la magia OOV.
- **Esperar que fastText "entienda" semántica de un neologismo arbitrario.** Reconstruye por forma; si los n-gramas no comparten semántica, el vecino será morfológico, no semántico.
- **Memoria.** `cc.es.300.bin` es grande. Para la kata puedes entrenar un fastText pequeño con `fasttext.train_unsupervised("corpus.txt", model='skipgram', minn=3, maxn=6, dim=100)`.

Variantes y extensiones

1. Entrena dos modelos sobre tu corpus: `skipgram` con subword (`minn=3,maxn=6`) y otro con `minn=0,maxn=0` (equivale a Word2Vec, sin subword). Compara su comportamiento ante OOV.
2. Mide similitud de pares morfológicos (`niño`–`niña`, `corro`–`corremos`) en fastText vs Word2Vec: fastText los acerca por n-gramas compartidos.
3. Evalúa el efecto de `minn` / `maxn`: n-gramas más largos = más específicos pero más OOV.

Anti-patrón

> ***Usar GloVe/Word2Vec en dominios de vocabulario abierto*** *(redes sociales, biomédico, español flexivo, logs)* ***y resignarse a OOV masivo*** *o a tratar todo lo no visto como* `<unk>`*. Si tu texto tiene neologismos, jerga, typos o morfología rica, fastText (o un tokenizador subpalabra + embeddings, como en Transformers) es la respuesta.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
OOV resuelto	fastText devuelve vecinos razonables para una palabra no vista o un typo
Contraste explícito	Muestras (o argumentas con KeyError) que palabra-completa no puede
Morfología	Evidencias que formas flexivas comparten vecindad por n-gramas
Conexión	Relacionas los n-gramas de fastText con la tokenización subpalabra (Kata 01)

Reflexión

1. ¿En qué se parecen los n-gramas de carácter de fastText a las fusiones de BPE? ¿En qué se diferencian (fastText *suma* n-gramas fijos; BPE *segmenta* greedy)?
2. ¿Por qué fastText puede dar un vector hasta para `xyzqwk`? ¿Es eso bueno o peligroso?
3. GloVe usa estadística **global**; Word2Vec, **local** (ventana). ¿Qué ventaja da cada una?

Referencias

- Pennington, Socher, Manning (2014). *GloVe: Global Vectors for Word Representation.* EMNLP.
- Bojanowski, Grave, Joulin, Mikolov (2017). *Enriching Word Vectors with Subword Information.* TACL (fastText).

Kata 08 — Clasificación: TF-IDF + Regresión Logística vs embeddings promediados

Objetivo. Medir, con validación cruzada, **cuándo** los embeddings densos superan a TF-IDF en una tarea real de clasificación de texto — y descubrir que el baseline lineal es sorprendentemente difícil de batir. Base: `semantica/regresi_n_logistica_b_sica_2.ipynb`.

Teoría formal

Regresión logística para clasificación binaria: modela `P(y=1|x) = σ(w·x + b)` con `σ` la sigmoide, y ajusta `w, b` minimizando la **entropía cruzada**:

```
J = − (1/n) Σ [ yᵢ log pᵢ + (1−yᵢ) log(1−pᵢ) ]      con pᵢ = σ(w·xᵢ + b)
```

Sobre vectores TF-IDF es un baseline brutal en clasificación de texto: lineal, convexo (óptimo global), rápido, y **interpretable** — el peso `w_t` de cada término dice cuánto empuja hacia una clase. Captura "términos discriminativos" exactos (p. ej. la palabra "reembolso" para detectar quejas).

Embeddings promediados (mean-pooling). Representas un documento como el **promedio** de los vectores (Word2Vec/fastText) de sus palabras → un vector denso de 100–300 dims. Captura semántica suave (sinónimos cuentan parecido) pero **pierde** la señal de términos exactos y el orden. No siempre gana: en datasets chicos o con jerga muy discriminativa, TF-IDF lo supera.

La lección de arquitecto: **mide, no asumas.** "Embeddings = mejor" es un mito; depende del tamaño del dataset, el dominio y si la tarea se decide por términos exactos o por tema general.

Implementación completa

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

# textos: lista de str ; y: etiquetas
# (usa tu dataset de la actividad; aquí asumimos `textos`, `y` cargados)

# --- Modelo A: TF-IDF + LogReg ---
modelo_A = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
    LogisticRegression(max_iter=1000, C=1.0),
)
f1_A = cross_val_score(modelo_A, textos, y, cv=5, scoring="f1_macro")

# --- Modelo B: embeddings fastText promediados + LogReg ---
import fasttext
ft = fasttext.load_model("cc.es.300.bin")
def doc_vector(texto):
    palabras = texto.lower().split()
    vs = [ft.get_word_vector(w) for w in palabras] or [np.zeros(300)]
    return np.mean(vs, axis=0)
X_emb = np.vstack([doc_vector(t) for t in textos])
f1_B     =     cross_val_score(LogisticRegression(max_iter=1000),     X_emb,     y,     cv=5,
        scoring="f1_macro")

print(f"TF-IDF + LogReg   F1-macro = {f1_A.mean():.3f} ± {f1_A.std():.3f}")
print(f"fastText avg+LogReg F1-macro = {f1_B.mean():.3f} ± {f1_B.std():.3f}")

# Interpretabilidad de A: términos más discriminativos
modelo_A.fit(textos, y)
vec = modelo_A.named_steps["tfidfvectorizer"]
clf = modelo_A.named_steps["logisticregression"]
terms = np.array(vec.get_feature_names_out())
coef = clf.coef_[0]
print("\n+ clase:", terms[coef.argsort()[::-1][:8]].tolist())
print("- clase:", terms[coef.argsort()[:8]].tolist())
```

Salida esperada (forma)

```
TF-IDF + LogReg     F1-macro = 0.882 ± 0.021
fastText avg+LogReg F1-macro = 0.851 ± 0.018

+ clase: ['excelente','recomiendo','perfecto','genial', ...]
- clase: ['pésimo','horrible','devolución','estafa', ...]
```

Muy a menudo TF-IDF gana (o empata) en datasets pequeños/medianos: los términos exactos son fuertes señales. La interpretabilidad es un bonus enorme — ves *por qué* clasifica así.

Errores comunes / debugging

- **Fugas de datos (leakage):** ajustar el `TfidfVectorizer` sobre todo el dataset antes del CV. Usa `Pipeline` para que el vectorizador se ajuste **dentro** de cada fold.
- **Mean-pooling con stopwords:** promediar incluyendo "de/la" diluye la señal. Filtra o pondera por IDF.
- **Comparar accuracy en clases desbalanceadas.** Usa `f1_macro` o `balanced_accuracy`.
- **No fijar `cv` ni semillas** → comparas ruido. Usa el mismo split para ambos modelos.

Variantes y extensiones

1. **Mean-pooling ponderado por TF-IDF** (en vez de promedio simple): suele cerrar parte de la brecha con TF-IDF puro. Combina lo mejor de ambos.
2. Concatena `[TF-IDF reducido por SVD || embedding promedio]` y entrena LogReg sobre la unión: a veces supera a ambos.
3. Curva de aprendizaje: repite con 10%, 50%, 100% de los datos. Verás que los embeddings tienden a alcanzar/ superar a TF-IDF **a medida que crecen los datos**.

Anti-patrón

> ***"Embeddings = siempre mejor".*** *Falso. En datasets pequeños, dominios con jerga discriminativa, o tareas decididas por términos exactos, TF-IDF + LogReg gana — y encima es interpretable y barato. Elegir el modelo complejo sin medir contra el baseline simple es mala ingeniería.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
CV sin leakage	Vectorizador ajustado dentro de cada fold (Pipeline)
Métrica robusta	F1-macro (o balanced) con media ± std de ambos modelos
Interpretabilidad	Listas los términos más discriminativos del modelo TF-IDF
Explicación del ganador	Justificas por qué ganó cada uno **en este dataset concreto**

Reflexión

1. ¿Qué información conserva TF-IDF que el mean-pooling de embeddings destruye? (orden, términos exactos, especificidad.)

2. ¿En qué condiciones esperarías que los embeddings superen claramente a TF-IDF?
3. ¿Por qué la regresión logística es un baseline tan duro de batir en texto, y cuándo conviene saltar a un modelo no lineal o a fine-tuning de BERT (Kata 09)?

Referencias

- Jurafsky & Martin, cap. 5 (Logistic Regression).
- Wang & Manning (2012). *Baselines and Bigrams: Simple, Good Sentiment and Topic Classification.* (por qué los baselines lineales son tan fuertes.)
- Notebook base: `semantica/regresi_n_logistica_b_sica_2.ipynb`.

Bloque D — Embeddings contextuales y de oración

Word2Vec, GloVe y fastText comparten una limitación de fondo: son **estáticos**. Cada palabra tiene **un** vector, fijo, sin importar la frase. `banco` (dinero) y `banco` (asiento) colapsan en el mismo punto. Los modelos contextuales (ELMo, BERT, BETO, GPT) rompen eso: el vector de un token **depende de toda la oración** vía el mecanismo de atención. Y los modelos de oración (SBERT) llevan la idea al nivel de frase entera para permitir búsqueda semántica escalable. Aquí se cierra el continuum.

Kata 09 — Polisemia con BERT/BETO: "banco" no es un solo vector

Objetivo. Demostrar empíricamente que un embedding **contextual** asigna vectores **distintos** a la misma palabra según la frase — algo imposible para Word2Vec — y medirlo con coseno. Base: `embeddings/taller7_solucion.ipynb`.

Teoría formal

BERT (Devlin et al., 2018) es un Transformer **encoder** entrenado con *Masked Language Modeling*: se ocultan ~15% de los tokens y el modelo los predice usando **todo** el contexto bidireccional. La pieza clave es la **autoatención**: el vector de cada token en cada capa es una combinación ponderada de los vectores de *todos* los tokens de la secuencia. Por eso `banco` rodeado de {dinero, crédito} produce un vector distinto que rodeado de {parque, sentarse}: la atención mezcla el contexto en la representación.

- **BETO** = BERT entrenado en español (`dccuchile/bert-base-spanish-wwm-cased`). Para texto en español rinde mejor que un BERT multilingüe genérico.
- **Tokenización WordPiece** (`##`): palabras se parten en subpalabras. Si `banco` se parte (no suele, es frecuente), debes decidir cómo agregar los subtokens (promedio, primer subtoken...). La elección afecta el resultado.
- **De qué capa sacar el embedding:** la última capa es muy específica de la tarea de pre-entrenamiento; capas intermedias (o promedio de las últimas 4) suelen dar mejores representaciones de palabra. Para esta kata, la última capa basta para ver el efecto.

Implementación completa

```
from transformers import AutoTokenizer, AutoModel
import torch, torch.nn.functional as F

NOMBRE = "dccuchile/bert-base-spanish-wwm-cased"   # BETO
tok = AutoTokenizer.from_pretrained(NOMBRE)
model = AutoModel.from_pretrained(NOMBRE).eval()

frases = [
    "Saqué dinero del banco esta mañana.",          # 0: financiero
    "El banco aprobó mi solicitud de crédito.",     # 1: financiero
    "Me senté en el banco del parque a leer.",      # 2: mueble/asiento
]

@torch.no_grad()
def embedding_palabra(frase, palabra):
    enc = tok(frase, return_tensors="pt")
    out = model(**enc).last_hidden_state[0]          # (n_tokens, 768)
    ids = enc.input_ids[0]
    objetivo = tok.convert_tokens_to_ids(palabra)
    pos = (ids == objetivo).nonzero(as_tuple=True)[0]
    if len(pos) == 0:                                # palabra partida en subtokens
        sub = tok.tokenize(frase)
        idxs = [i + 1 for i, t in enumerate(sub) if palabra in t.replace("##", "")]
        return out[idxs].mean(0)
    return out[pos[0]]

v = [embedding_palabra(f, "banco") for f in frases]
print("financiero vs financiero :", F.cosine_similarity(v[0], v[1], dim=0).item())
print("financiero vs asiento    :", F.cosine_similarity(v[0], v[2], dim=0).item())
print("financiero2 vs asiento   :", F.cosine_similarity(v[1], v[2], dim=0).item())
```

Salida esperada (forma)

```
financiero vs financiero : 0.78
financiero vs asiento    : 0.55
financiero2 vs asiento   : 0.53
```

Los dos sentidos financieros tienen coseno **claramente mayor** entre sí que cualquiera contra el sentido "asiento". Con Word2Vec los tres serían **idénticos** (un solo vector de `banco`): coseno 1.0 en los tres casos. Esa diferencia ES la kata.

Errores comunes / debugging

- **Olvidar** `model.eval()` **y** `torch.no_grad()` → dropout activo (resultados ruidosos) y consumo de memoria por el grafo de gradientes.
- **Indexar mal por el** `[CLS]` / `[SEP]`. El tokenizador añade tokens especiales al inicio/fin; el `+1` y el manejo de posiciones deben contarlos.
- **Palabra partida en WordPiece** sin agregación → tomas solo medio token. El código promedia los subtokens.
- **Comparar mayúsculas/acentos** distintos a los del tokenizer (`cased`): "Banco" ≠ "banco".

Variantes y extensiones

1. Repite con embeddings **estáticos** (Word2Vec/fastText) y muestra que los tres cosenos son 1.0 (o iguales): el contraste hace tangible "estático vs contextual".
2. Promedia las **últimas 4 capas** en vez de la última y mira si la separación de sentidos mejora.
3. Mini-WSD: dadas 10 frases con `banco`, agrupa por coseno (clustering) y comprueba que emergen 2 clusters (dinero / asiento) sin etiquetas.

Anti-patrón

> ***Usar embeddings estáticos para tareas que dependen del sentido en contexto*** *— desambiguación (WSD), detección de sarcasmo, negación, correferencia. Por diseño no pueden: colapsan todos los sentidos de una palabra en un punto. Para eso existen los contextuales.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Vectores distintos	El mismo token `banco` produce vectores distintos por frase
Medición correcta	Coseno(mismo sentido) > coseno(sentidos distintos), reportado
Contraste estático	Muestras que Word2Vec daría coseno idéntico en los tres casos
Manejo de subtokens	El código agrega correctamente si la palabra se parte en WordPiece

Reflexión

1. Si `banco` se parte en subtokens, ¿conviene promediar, usar el primer subtoken, o el último? ¿Cómo afecta al coseno?
2. ¿Por qué la **última** capa de BERT no siempre da el mejor embedding de palabra? ¿Qué pasa en capas intermedias?
3. BERT da embeddings de **token** geniales pero malos de **oración** si promedias. ¿Por qué? — lo resuelve la Kata 10.

Referencias

- Devlin, Chang, Lee, Toutanova (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL.
- Cañete et al. (2020). *Spanish Pre-Trained BERT Model and Evaluation Data* (BETO).

- Notebook base: `embeddings/taller7_solucion.ipynb`.

Kata 10 — Búsqueda semántica con SBERT / BioBERT

Objetivo. Construir un mini-buscador semántico: dada una consulta en lenguaje natural, recuperar las oraciones más cercanas de un corpus por coseno de **embeddings de oración**. Cierra el continuum (de caracteres a recuperación semántica) usando el dataset `embeddings/Biobert`.

Contexto y motivación

Promediar los embeddings de token de BERT da vectores de oración **mediocres** para comparar (la última capa está optimizada para MLM, no para que el coseno entre oraciones tenga sentido). **Sentence-BERT** (Reimers & Gurevych, 2019) lo arregla: afina BERT con una arquitectura **siamesa** (dos copias con pesos compartidos) y una función de pérdida que **empuja el coseno** entre oraciones similares hacia 1 y entre disímiles hacia 0. Resultado: embeddings de oración donde el coseno **sí** mide similitud semántica → permite búsqueda escalable (codificas el corpus una vez, luego cada query es un producto matriz–vector).

Teoría formal

- **Arquitectura siamesa + pooling.** Ambas oraciones pasan por el mismo encoder BERT; se hace *mean-pooling* de los tokens (ponderado por la máscara de atención) → un vector por oración.
- **Objetivos de entrenamiento.** Según los datos: (a) **clasificación** sobre NLI con `[u; v; |u−v|]`; (b) **regresión** que minimiza `MSE(cos(u,v), score_humano)` en datasets STS; (c) **triplet loss** `max(0, ‖a−p‖ − ‖a−n‖ + margen)` que acerca el ancla `a` a su positivo `p` y la aleja del negativo `n`.
- **BioBERT / S-BioBERT.** BERT (o SBERT) pre-entrenado en literatura biomédica (PubMed). En dominio médico supera ampliamente a BERT genérico porque el vocabulario y las co-ocurrencias (genes, fármacos, patologías) son específicos.
- **Retrieval.** Pipeline: `encode(corpus)` una vez → normaliza L2 → `encode(query)` → `top-k` por coseno (= producto interno sobre normalizados, ver Kata 04). Escala a millones con un índice ANN (FAISS).

Implementación completa

```
from sentence_transformers import SentenceTransformer, util

# SBERT afinado en dominio biomédico (NLI+STS sobre BioBERT)
modelo = SentenceTransformer("pritamdeka/S-BioBert-snli-multinli-stsb")

# corpus: una oración/abstract por línea
corpus = [l.strip() for l in open("embeddings/Biobert/abstracts.txt", encoding="utf-8")
          if l.strip()]

# 1) codificar el corpus UNA vez, normalizado L2
emb_corpus = modelo.encode(corpus, convert_to_tensor=True,
                           normalize_embeddings=True, show_progress_bar=True)

# 2) consultar
consultas = [
    "treatment of type 2 diabetes with metformin",
    "side effects of chemotherapy in breast cancer",
]
for q in consultas:
    eq = modelo.encode(q, convert_to_tensor=True, normalize_embeddings=True)
    hits = util.semantic_search(eq, emb_corpus, top_k=5)[0]
    print(f"\n🔎 {q}")
    for h in hits:
        print(f"   {h['score']:.3f}  {corpus[h['corpus_id']][:90]}")
```

Salida esperada (forma)

```
🔎 treatment of type 2 diabetes with metformin
   0.71  Metformin remains the first-line pharmacologic therapy for type 2 diabetes ...
   0.63  Glycemic control with biguanides reduces cardiovascular risk in ...
   0.55  Insulin sensitizers improve outcomes in patients with ...
   ...
```

Validación: los top-k son temáticamente relevantes y el score **decrece de forma coherente** al bajar en el ranking. Nota que el primer resultado puede **no compartir las palabras exactas** de la query ("metformin"/"biguanides") — eso es la diferencia con TF-IDF: recupera por **significado**, no por coincidencia léxica.

Errores comunes / debugging

- **Mean-pooling de BERT base** y esperar buen ranking. Usa un modelo SBERT entrenado para similitud; BERT crudo no sirve para coseno de oración.
- **BERT genérico en texto biomédico.** Pierde frente a BioBERT por vocabulario. Usa el modelo de dominio.
- **No normalizar.** Si no pasas `normalize_embeddings=True`, mezclas magnitudes (ver Kata 04).
- **Re-codificar el corpus en cada query.** Codifícalo una vez y reutiliza el tensor; para corpus grandes, persiste los embeddings e indexa con FAISS.

Variantes y extensiones

1. **Comparación de baselines:** mismo corpus y queries con TF-IDF (Kata 03). Muestra una query donde TF-IDF falla (sinónimos sin solapamiento léxico) y SBERT acierta.
2. **Re-ranking cross-encoder:** recupera top-50 con SBERT (bi-encoder, rápido) y reordena con un cross-encoder (más preciso, lento). Patrón estándar de retrieval moderno.
3. **Índice FAISS** sobre los embeddings normalizados (`IndexFlatIP`) y mide el speedup vs búsqueda exhaustiva.
4. **RAG mínimo:** usa los top-k recuperados como contexto para responder una pregunta — conecta esta kata con sistemas de generación aumentada por recuperación.

Anti-patrón

> ***Promediar tokens de BERT base para comparar oraciones**, o usar **BERT genérico en un dominio especializado** (biomédico, legal). Para similitud de oración necesitas un modelo entrenado para ello (SBERT); para un dominio, uno pre-entrenado en ese dominio (BioBERT). Saltarse esto da rankings que parecen funcionar en ejemplos fáciles y fallan en los difíciles.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Pipeline correcto	Corpus codificado una vez, normalizado, query → top-k por coseno
Relevancia	Top-5 temáticamente pertinentes; scores decrecen coherentemente
Semántica > léxico	Muestras un acierto sin solapamiento de palabras exactas
Modelo adecuado	Usas SBERT (no BERT crudo) y, para biomédico, un modelo de dominio

Reflexión

1. ¿Por qué el mean-pooling de BERT base da malos vectores de oración pese a dar buenos de token? ¿Qué cambia SBERT con su entrenamiento siamés?
2. Cierra el continuum: nombra qué pregunta resuelve **cada** etapa, de la Kata 01 (¿cómo partir?) a la Kata 10 (¿cómo recuperar por significado?).
3. ¿Cuándo un bi-encoder (SBERT) basta y cuándo necesitas un cross-encoder de re-ranking?

Referencias

- Reimers & Gurevych (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP.
- Lee et al. (2020). *BioBERT: a pre-trained biomedical language representation model.* Bioinformatics.

- Dataset base: `embeddings/Biobert`, notebook `embeddings/taller7_solucion.ipynb`.

Bloque E — Del RNN al Transformer (la arquitectura por debajo)

Los Bloques A–D te llevaron de "cómo partir el texto" a "embeddings de oración". Pero las katas 09–10 usaron BERT y SBERT como **cajas negras**. Este bloque abre la caja: explica la **arquitectura** que produce los embeddings contextuales, y —crucial para un arquitecto— *por qué* surgió. Toda tecnología se entiende mejor contra el problema que vino a resolver.

La historia en una página: el predecesor de los Transformers

Antes de 2017, el procesamiento de secuencias en PLN se hacía con **redes recurrentes (RNN)** y sus variantes **LSTM** y **GRU**. La idea: leer la secuencia **token por token**, manteniendo un **estado oculto** h_t que resume todo lo visto hasta t:

$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$

Para tareas de secuencia-a-secuencia (traducción, resumen) se usaba la arquitectura **encoder-decoder (seq2seq)** (Sutskever et al., 2014): un RNN *encoder* comprime toda la oración de entrada en **un solo vector** (el último estado oculto), y un RNN *decoder* genera la salida a partir de ese vector. Tres problemas estructurales lo hundían:

1. **El cuello de botella del vector único.** Toda la oración —corta o de 80 palabras— debe caber en un vector de tamaño fijo. La información del inicio se "olvida" al llegar al final. La calidad de traducción caía en picado con la longitud.
2. **Gradientes que se desvanecen/explotan.** Al propagar el error hacia atrás a través de muchos pasos temporales, el gradiente se multiplica repetidamente → tiende a 0 (desvanece) o a ∞ (explota). Las RNN simples **no aprenden dependencias largas**. LSTM y GRU lo mitigan con **compuertas** (gates) que controlan qué recordar/olvidar, pero no lo eliminan.
3. **Imposible paralelizar.** h_t depende de h_{t-1} → el cómputo es **secuencial** por diseño. No se aprovecha el paralelismo masivo de las GPU. Entrenar con corpus enormes era lentísimo.

El puente: el mecanismo de atención (2014–2015). Bahdanau et al. (2014) y Luong et al. (2015) atacaron el problema #1: en vez de comprimir todo en un vector, dejaron que el decoder "**mire**" **todos los estados del encoder** y pondere cuáles son relevantes en cada paso de generación. Esa ponderación —los **pesos de atención**— resolvió el cuello de botella y, de paso, dio interpretabilidad (se ve qué palabra de origen alinea con cada palabra de destino). La atención era todavía un *complemento* del RNN.

El salto: "Attention Is All You Need" (Vaswani et al., 2017). La pregunta provocadora: si la atención es lo que funciona, ¿necesitamos el RNN? Respuesta: **no**. El **Transformer** elimina la recurrencia por completo y se construye solo con **auto-atención** (self-attention)

- capas feed-forward. Consecuencias:
- **Paralelización total**: todos los tokens se procesan a la vez (no hay dependencia secuencial) → entrenable en corpus gigantes con GPU/TPU.
- **Dependencias largas en O(1) pasos**: cualquier token puede atender directamente a cualquier otro, sin importar la distancia (a diferencia de los O(n) pasos del RNN).
- Coste: la atención es **$O(n^2)$** en memoria/cómputo respecto a la longitud n (cada token mira a todos) → de ahí la obsesión por tokenizar eficiente (¡Bloque A!) y por variantes de atención eficiente.

BERT (Kata 09) es la pila **encoder** del Transformer; GPT es la pila **decoder**; T5 usa ambas. Las katas 11–15 reconstruyen esta historia con las manos.

Self-attention en una fórmula (la usarás en las katas 12–13)

Cada token se proyecta en tres vectores: **Query** (qué busco), **Key** (qué ofrezco), **Value** (qué entrego). La atención escalada por producto punto (Vaswani et al., 2017):

```
Attention(Q, K, V) = softmax( Q·Kᵀ / √d_k ) · V
```

- `Q·Kᵀ`: similitud (producto punto) de cada query con cada key → matriz `n×n` de scores.
- `/ √d_k`: escala por la raíz de la dimensión de las keys; sin ella, con `d_k` grande los productos crecen, el softmax se satura y los gradientes mueren.
- `softmax` por filas: convierte scores en pesos que suman 1 → "cuánta atención presta el token i a cada token j".
- `· V`: cada token recibe una mezcla ponderada de los values de todos → su nueva representación, **contextual**.

Multi-head: se hace `h` veces en paralelo con proyecciones distintas (cada "cabeza" aprende un tipo de relación: sintáctica, correferencia, etc.), y se concatena. Eso es lo que da a BERT su riqueza contextual de la Kata 09.

Kata 11 — RNN/LSTM de carácter desde cero: ver el cuello de botella

Objetivo. Implementar un modelo de lenguaje a nivel de carácter con una RNN (y luego LSTM) para *sentir* en las manos la recurrencia, el estado oculto y por qué los gradientes se desvanecen en dependencias largas.

Contexto y motivación

Antes de criticar a las RNN hay que haberlas programado. Un modelo de lenguaje de carácter (predecir el siguiente carácter dado el contexto) es el "hola mundo" de las secuencias: con ~100 líneas ves el estado oculto propagándose, generas texto muestreando, y observas que la RNN simple olvida lo lejano. Es el famoso ejercicio de Karpathy ("char-rnn").

Teoría formal

RNN simple, paso a paso sobre la secuencia $x_1 \dots x_T$:

```
h_t = tanh(W_hh·h_{t-1} + W_xh·x_t + b_h)        # estado oculto (memoria)
y_t = W_hy·h_t + b_y                               # logits del siguiente token
p̂_t = softmax(y_t)
```

Entrenamiento por **Backpropagation Through Time (BPTT)**: se "desenrolla" la red en T pasos y se propaga el gradiente hacia atrás. El gradiente de la pérdida respecto a un estado lejano incluye un producto de T jacobianos $\partial h_t / \partial h_{t-1}$. Si los valores propios de W_{hh} son <1, ese producto $\to \mathbf{0}$ (vanishing); si >1, $\to \infty$ (exploding). Por eso la RNN simple no captura "el sujeto del inicio concuerda con el verbo del final".

LSTM (Hochreiter & Schmidhuber, 1997) añade una **celda de memoria** c_t y tres compuertas (olvido f, entrada i, salida o) que regulan el flujo. La clave: c_t se actualiza de forma **aditiva** ($c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$), creando una "autopista" por la que el gradiente fluye sin multiplicarse repetidamente → mitiga el desvanecimiento.

Implementación completa (RNN char-level, numpy)

```
import numpy as np

data = open("guia_pdf/guia.md", encoding="utf-8").read()
chars = sorted(set(data))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
V = len(chars)
H, seq_len, lr = 128, 25, 1e-1

Wxh = np.random.randn(H, V) * 0.01
Whh = np.random.randn(H, H) * 0.01
Why = np.random.randn(V, H) * 0.01
bh, by = np.zeros((H, 1)), np.zeros((V, 1))

def loss_and_grads(inputs, targets, hprev):
    xs, hs, ps = {}, {-1: hprev}, {}
    loss = 0
    for t in range(len(inputs)):                        # FORWARD (secuencial!)
        xs[t] = np.zeros((V, 1)); xs[t][inputs[t]] = 1
        hs[t] = np.tanh(Wxh @ xs[t] + Whh @ hs[t-1] + bh)
        y = Why @ hs[t] + by
        ps[t] = np.exp(y) / np.sum(np.exp(y))
        loss += -np.log(ps[t][targets[t], 0])
    dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
    dbh, dby, dhnext = np.zeros_like(bh), np.zeros_like(by), np.zeros_like(hs[0])
    for t in reversed(range(len(inputs))):              # BPTT
        dy = np.copy(ps[t]); dy[targets[t]] -= 1
        dWhy += dy @ hs[t].T; dby += dy
        dh = Why.T @ dy + dhnext
        dhraw = (1 - hs[t] ** 2) * dh                   # gradiente a través de tanh
        dbh += dhraw; dWxh += dhraw @ xs[t].T; dWhh += dhraw @ hs[t-1].T
        dhnext = Whh.T @ dhraw
    for d in (dWxh, dWhh, dWhy, dbh, dby):
        np.clip(d, -5, 5, out=d)                        # clip: frena el exploding
    return loss, dWxh, dWhh, dWhy, dbh, dby, hs[len(inputs)-1]

def sample(h, seed_ix, n):
    x = np.zeros((V, 1)); x[seed_ix] = 1; out = []
    for _ in range(n):
        h = np.tanh(Wxh @ x + Whh @ h + bh)
        p = np.exp(Why @ h + by); p /= p.sum()
        ix = np.random.choice(range(V), p=p.ravel())
        x = np.zeros((V, 1)); x[ix] = 1; out.append(ix)
    return "".join(itos[i] for i in out)

n, p = 0, 0
mWxh, mWhh, mWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
mbh, mby = np.zeros_like(bh), np.zeros_like(by)
hprev = np.zeros((H, 1))
while n < 20000:
    if p + seq_len + 1 >= len(data) or n == 0:
        hprev = np.zeros((H, 1)); p = 0
    inputs = [stoi[c] for c in data[p:p+seq_len]]
    targets = [stoi[c] for c in data[p+1:p+seq_len+1]]
    loss, dWxh, dWhh, dWhy, dbh, dby, hprev = loss_and_grads(inputs, targets, hprev)
    for param, dparam, mem in [(Wxh,dWxh,mWxh),(Whh,dWhh,mWhh),(Why,dWhy,mWhy),
                               (bh,dbh,mbh),(by,dby,mby)]:
        mem += dparam * dparam                          # Adagrad
        param += -lr * dparam / np.sqrt(mem + 1e-8)
    if n % 2000 == 0:
```

```
        print(f"iter {n}, loss {loss:.2f}\n----\n{sample(hprev, inputs[0], 200)}\n----")
    p += seq_len; n += 1
```

Salida esperada (forma)

```
iter 0,    loss 110.2   ----  óÑz9k...basura aleatoria...
iter 8000, loss 48.3    ----  ...la tokenización de la palabra es un...
```

La loss baja y el texto muestreado pasa de ruido a fragmentos pseudo-español. Pero observa: genera **localmente** plausible y **globalmente** incoherente — es el límite de la RNN simple.

Errores comunes / debugging

- **No hacer clip de gradientes** → loss salta a `NaN` (exploding). El `np.clip(-5,5)` lo evita.
- **Reiniciar `hprev` cada minibatch** → pierdes el contexto entre ventanas. Solo resetea al volver al inicio del corpus.
- **`lr` alto sin Adagrad** → diverge. Adagrad adapta el paso por parámetro.
- **One-hot mal indexado** → `targets[t]` debe ser índice escalar.

Variantes y extensiones

1. Reemplaza la RNN por una **LSTM** (`torch.nn.LSTM`) y compara: la LSTM mantiene coherencia en secuencias más largas gracias a la celda de memoria.
2. Mide la **norma del gradiente** respecto al primer paso temporal conforme crece `seq_len`: verás cómo se desvanece — el vanishing gradient hecho número.
3. Implementa **GRU** y compara nº de parámetros y calidad frente a LSTM.

Anti-patrón

> ***Creer que "más estado oculto" arregla la memoria larga.*** *No: el problema es el producto de jacobianos en BPTT, no el tamaño de `H`. La solución arquitectónica fue LSTM/GRU (rutas aditivas) y, definitivamente, la atención (acceso directo a todos los pasos). Subir `H` solo sube el costo.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Loss decreciente	La pérdida baja y el muestreo mejora de ruido a pseudo-texto
BPTT correcto	El backward refleja el desenrollado en el tiempo + clip
Estado propagado	`hprev` se conserva entre ventanas contiguas
Diagnóstico	Explicas, con tu propio output, el límite de coherencia global

Reflexión

1. ¿Por qué la actualización **aditiva** de la celda LSTM ayuda al gradiente frente a la **multiplicativa** de la RNN?
2. ¿Por qué este modelo no se puede paralelizar en el tiempo, y qué implica para entrenar con corpus de miles de millones de tokens?
3. Conecta con la Kata 13: ¿cómo resuelve la self-attention el acceso a tokens lejanos sin recurrencia?

Referencias

- Hochreiter & Schmidhuber (1997). *Long Short-Term Memory.*
- Sutskever, Vinyals, Le (2014). *Sequence to Sequence Learning with Neural Networks.*
- Karpathy (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks* (char-rnn).

Kata 12 — Atención de Bahdanau/Luong desde cero: el puente

Objetivo. Implementar el mecanismo de atención que rescató al seq2seq, y **visualizar la matriz de alineación** para ver cómo cada palabra generada "mira" las palabras de origen.

Contexto y motivación

Esta es la kata bisagra de toda la historia: la atención nació como parche al cuello de botella del RNN (Kata 11) y terminó siendo la idea que mató al RNN (Kata 13). Entenderla en su forma original (sobre un encoder RNN) hace trivial entender la self-attention después.

Teoría formal

En seq2seq con atención, el decoder en el paso `t` (estado s_t) no usa un único vector de contexto fijo, sino uno **dinámico** c_t que pondera todos los estados del encoder $h_1 \dots h_n$:

```
e_tj = score(s_{t-1}, h_j)          # afinidad del paso t con la palabra origen j
α_tj = softmax_j(e_tj)               # pesos de atención (suman 1 sobre j)
c_t  = Σ_j α_tj · h_j                # contexto: mezcla ponderada de estados origen
```

Dos formas clásicas del `score`:

- **Bahdanau (aditiva, 2014):** $\text{score} = v^\top \cdot \tanh(W_1 \cdot s_{t-1} + W_2 \cdot h_j)$ — una pequeña red.
- **Luong (multiplicativa, 2015):** $\text{score} = s_{t-1}^\top \cdot W \cdot h_j$ (o el producto punto directo $s_{t-1}^\top \cdot h_j$). Más simple y barata — y es la semilla del producto punto de los Transformers.

La matriz α (destino × origen) es la **alineación**: en traducción, revela qué palabra fuente corresponde a cada palabra generada.

Ejemplo numérico (intuición)

Traducir `"the cat"` → `"el gato"`. Al generar `"gato"`, el score de su estado contra `h_cat` es alto y contra `h_the` bajo → $\alpha(\text{gato, cat}) \approx 0.9$, $\alpha(\text{gato, the}) \approx 0.1$. El contexto `c` es casi `h_cat`. La atención **alinea** "gato" con "cat" sin que nadie se lo dijera.

Implementación completa (atención de Luong, numpy)

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

# estados del encoder (n_origen x d) y del decoder (m_destino x d), simulados
np.random.seed(0)
n, m, d = 5, 4, 8                       # 5 palabras origen, 4 destino, dim 8
H_enc = np.random.randn(n, d)           # h₁..h₅
S_dec = np.random.randn(m, d)           # s₁..s₄

# Luong "general": score = S · W · Hᵀ
W = np.random.randn(d, d) * 0.1
scores = S_dec @ W @ H_enc.T            # (m x n): afinidad destino-origen
alpha  = softmax(scores, axis=1)        # pesos por fila (cada destino reparte 1.0)
context = alpha @ H_enc                 # (m x d): contexto dinámico por paso destino

print("Matriz de alineación α (destino × origen):")
print(np.round(alpha, 2))
print("Suma por fila (debe ser 1):", np.round(alpha.sum(1), 3))

# visualización opcional
import matplotlib.pyplot as plt
plt.imshow(alpha, cmap="viridis"); plt.xlabel("origen"); plt.ylabel("destino")
plt.colorbar(); plt.title("Pesos de atención"); plt.savefig("kata12_align.png")
```

Salida esperada (forma)

```
Matriz de alineación α (destino × origen):
[[0.31 0.12 0.08 0.27 0.22]
 [0.10 0.44 0.18 0.15 0.13]
 ...]
Suma por fila (debe ser 1): [1. 1. 1. 1.]
```

Cada fila suma 1 (es una distribución sobre las palabras origen). El mapa de calor muestra, para cada palabra generada, dónde puso la atención. En un modelo entrenado de verdad, esa diagonal-ish revela la alineación de traducción.

Errores comunes / debugging

- **Softmax por el eje equivocado.** Debe normalizar sobre **origen** (axis=1), no destino. Si las columnas suman 1 en vez de las filas, está al revés.
- **Olvidar restar el máximo en softmax** → overflow numérico con scores grandes.
- **Confundir contexto con estado.** c_t se concatena/combina con s_t para predecir; no lo reemplaza.

Variantes y extensiones

1. Implementa la versión **aditiva de Bahdanau** y compara los pesos con la multiplicativa de Luong sobre los mismos estados.
2. Añade **masking**: en generación causal, un token no puede atender al futuro → pon `-inf` en los scores correspondientes antes del softmax (te prepara para la Kata 13).
3. Sustituye `S·W·Hᵀ` por el **producto punto puro** `S·Hᵀ/√d` y observa que ya es, esencialmente, la atención del Transformer (sin Q/K/V separados todavía).

Anti-patrón

> ***Tratar los pesos de atención como "explicación" causal del modelo.*** *Son una ponderación útil e interpretable, pero "atención alta" ≠ "esta es la razón de la predicción". Hay literatura entera (Attention is not Explanation / ...is not not Explanation) sobre este matiz. Úsalos para diagnóstico, no como prueba.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
α válida	Cada fila suma 1; dimensiones (destino × origen) correctas
Contexto dinámico	c_t es distinto por paso destino (no un vector fijo)
Visualización	Mapa de calor de alineación guardado
Conexión	Explicas cómo esto resuelve el cuello de botella de la Kata 11

Reflexión

1. ¿Por qué la atención elimina el cuello de botella del vector único del seq2seq?
2. ¿Qué ventaja computacional tiene el score multiplicativo (Luong) sobre el aditivo (Bahdanau)? (pista: es un producto matricial → GPU.)
3. En la Kata 13, ¿qué pasa cuando Query, Key y Value vienen **de la misma secuencia**?

Referencias

- Bahdanau, Cho, Bengio (2014). *Neural Machine Translation by Jointly Learning to Align and Translate.*
- Luong, Pham, Manning (2015). *Effective Approaches to Attention-based NMT.*

Kata 13 — Self-attention escalada + multi-head desde cero

Objetivo. Implementar el corazón del Transformer —scaled dot-product self-attention y multi-head— en numpy puro, y entender por qué reemplaza a la recurrencia.

Contexto y motivación

Si dominas esta kata, dominas el 80% del Transformer. Self-attention = la atención de la Kata 12 pero donde **Query, Key y Value salen de la misma secuencia** (cada token atiende a los demás tokens de su propia oración). No hay recurrencia: todos los tokens se procesan a la vez.

Teoría formal

Dada una secuencia de `n` tokens, cada uno embebido en `d` dimensiones (matriz `X` de `n×d`):

```
Q = X·W_Q      K = X·W_K      V = X·W_V          # tres proyecciones lineales
Attention(Q,K,V) = softmax( Q·Kᵀ / √d_k ) · V    # d_k = dim de las keys
```

- `Q·Kᵀ` (`n×n`): cuánto se parece la *pregunta* de cada token a la *clave* de cada otro.
- `/√d_k`: **escalado** imprescindible. Sin él, con `d_k` grande la varianza del producto punto crece $\sim$ `d_k`, el softmax se satura (casi one-hot) y su gradiente ≈ 0 → no aprende.
- `softmax` por filas → pesos de atención.
- `·V` → cada token recibe una mezcla de los *values* → su representación contextual.

Multi-head attention: en vez de una atención de dimensión `d`, se hacen `h` atenciones en paralelo de dimensión `d/h`, cada una con sus propias `W_Q,W_K,W_V`. Cada **cabeza** aprende relaciones distintas (una mira el verbo, otra la correferencia...). Se concatenan y se proyectan:

```
MultiHead(X) = Concat(head₁,…,head_h)·W_O,    head_i = Attention(X·W_Qⁱ, X·W_Kⁱ, X·W_Vⁱ)
```

Masking causal: en un decoder (GPT) un token no debe ver el futuro → se suma una máscara triangular de `-inf` a los scores antes del softmax.

Implementación completa (numpy)

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x); return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d_k)        # (..., n, n)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)             # tapa posiciones prohibidas
    attn = softmax(scores, axis=-1)
    return attn @ V, attn

def multi_head_attention(X, Wq, Wk, Wv, Wo, h):
    n, d = X.shape
    dk = d // h
    Q, K, V = X @ Wq, X @ Wk, X @ Wv                      # (n, d)
    # parte en h cabezas: (h, n, dk)
    split = lambda M: M.reshape(n, h, dk).transpose(1, 0, 2)
    Qh, Kh, Vh = split(Q), split(K), split(V)
    out, attn = scaled_dot_product_attention(Qh, Kh, Vh)  # (h, n, dk), (h, n, n)
    concat = out.transpose(1, 0, 2).reshape(n, d)         # vuelve a (n, d)
    return concat @ Wo, attn

if __name__ == "__main__":
    np.random.seed(0)
    n, d, h = 6, 16, 4                                    # 6 tokens, dim 16, 4 cabezas
    X = np.random.randn(n, d)
    Wq, Wk, Wv, Wo = (np.random.randn(d, d) * 0.1 for _ in range(4))
    salida, attn = multi_head_attention(X, Wq, Wk, Wv, Wo, h)
    print("salida:", salida.shape, "| atención por cabeza:", attn.shape)
    print("pesos cabeza 0 (token 0 atiende a):", np.round(attn[0, 0], 2),
          " suma:", round(attn[0, 0].sum(), 3))

    # demostración del por qué del √d_k: varianza del score con/sin escalar
    for dk in [8, 128, 1024]:
        q = np.random.randn(10000, dk); k = np.random.randn(10000, dk)
        raw = (q * k).sum(1)
                        print(f"d_k={dk:4d}      var(q·k)≈{raw.var():7.1f}      var(q·k/
        √d_k)≈{(raw/np.sqrt(dk)).var():.2f}")
```

Salida esperada (forma)

```
salida: (6, 16) | atención por cabeza: (4, 6, 6)
pesos cabeza 0 (token 0 atiende a): [0.18 0.2  0.13 0.17 0.15 0.17]  suma: 1.0

d_k=   8  var(q·k)≈    8.0  var(q·k/√d_k)≈1.00
d_k= 128  var(q·k)≈  128.3  var(q·k/√d_k)≈1.00
d_k=1024  var(q·k)≈ 1024.7  var(q·k/√d_k)≈1.00
```

La segunda parte **demuestra empíricamente** por qué se divide entre `√d_k`: sin escalar, la varianza del score crece con `d_k` (8, 128, 1024) → softmax saturado; al escalar, la varianza se mantiene ≈1 sin importar la dimensión.

Errores comunes / debugging

- **Olvidar el `/√d_k`** → con `d` grande el modelo no aprende (gradiente del softmax ≈0). Es el error que la segunda parte del código hace tangible.
- **Reshape/transpose mal en multi-head** → las cabezas se mezclan. Verifica que `attn` sea `(h, n, n)` y cada fila sume 1.
- **Máscara con `0` en vez de `-inf`** antes del softmax → el token "ve" el futuro igual (peso pequeño pero no nulo). Usa `-1e9`.
- **Self vs cross-attention:** en self, Q,K,V vienen del mismo `X`; en cross (decoder mirando encoder), Q viene del decoder y K,V del encoder.

Variantes y extensiones

1. Añade **máscara causal triangular** y verifica que el token `i` solo atiende a `≤ i` (matriz de atención triangular inferior).
2. Implementa el **bloque Transformer completo**: `LayerNorm(X + MultiHead(X))` seguido de `LayerNorm(· + FFN(·))` con conexión residual. Es literalmente una capa de BERT.
3. Cuenta operaciones: confirma que la atención es **O(n²·d)** → justifica por qué la longitud de secuencia (y por tanto el tokenizador, Bloque A) importa tanto para el costo.

Anti-patrón

> ***Quitar el escalado `√d_k`** ("es solo una constante") **o no usar multi-head** ("una atención basta"). El escalado es lo que mantiene los gradientes vivos en dimensión alta; las múltiples cabezas son lo que permite capturar relaciones heterogéneas en paralelo. Ambos no son adornos: son por qué el Transformer funciona.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Atención válida	Matriz `(h,n,n)`, cada fila suma 1
Escalado justificado	Demuestras numéricamente el efecto de `√d_k` en la varianza
Multi-head correcto	Split/concat de cabezas sin mezclar dimensiones
Self-attention	Q,K,V derivan del mismo `X`; explicas el contraste con cross-attention

Reflexión

1. ¿Por qué la self-attention captura dependencias largas en **O(1) pasos** mientras la RNN necesita O(n)? ¿A qué costo (pista: $O(n^2)$)?
2. ¿Qué información se pierde si NO añades codificación posicional? (abre la Kata 14.)
3. ¿Qué relación hay entre las "cabezas" y los distintos sentidos/relaciones que viste en la polisemia de la Kata 09?

Referencias

- Vaswani et al. (2017). *Attention Is All You Need.* NeurIPS.
- Alammar (2018). *The Illustrated Transformer* (visual, imprescindible).

Kata 14 — Codificación posicional sinusoidal

Objetivo. Entender por qué la self-attention **necesita** inyectar la posición de cada token, implementar la codificación posicional sinusoidal y visualizarla.

Contexto y motivación

La self-attention de la Kata 13 es **permutación-invariante**: si barajas los tokens de entrada, la salida (por token) es la misma barajada — la atención no "sabe" el orden. Pero el orden importa: "el perro mordió al hombre" ≠ "el hombre mordió al perro". Como no hay recurrencia que codifique posición implícitamente (como en el RNN), hay que **añadirla explícitamente**.

Teoría formal

El Transformer original suma a cada embedding de token un **vector de posición** de la misma dimensión `d`. La versión sinusoidal (Vaswani et al., 2017), para la posición `pos` y la dimensión `i`:

```
PE(pos, 2i)   = sin( pos / 10000^(2i/d) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d) )
```

Cada dimensión es una sinusoide de distinta frecuencia (de muy rápida a muy lenta). Tres propiedades buscadas:

- **Única por posición** pero **acotada** en [-1,1] (no explota con secuencias largas).
- **Distancias relativas codificables linealmente:** existe una matriz lineal que mapea `PE(pos)` a `PE(pos+k)` → el modelo puede aprender a atender "k posiciones a la izquierda" de forma uniforme.
- **Generaliza a longitudes no vistas** (al ser una función fija, no una tabla aprendida).

Alternativa moderna: **embeddings posicionales aprendidos** (BERT) — una tabla entrenable; funcionan bien pero no extrapolan más allá de la longitud máxima de entrenamiento. Variantes recientes: RoPE, ALiBi.

Implementación completa

```
import numpy as np
import matplotlib.pyplot as plt

def positional_encoding(max_len, d):
    pos = np.arange(max_len)[:, None]                      # (max_len, 1)
    i = np.arange(d)[None, :]                              # (1, d)
    angle = pos / np.power(10000, (2 * (i // 2)) / d)      # (max_len, d)
    pe = np.zeros((max_len, d))
    pe[:, 0::2] = np.sin(angle[:, 0::2])                   # dims pares: seno
    pe[:, 1::2] = np.cos(angle[:, 1::2])                   # dims impares: coseno
    return pe

PE = positional_encoding(max_len=100, d=64)
print("PE shape:", PE.shape, "| rango:", PE.min().round(2), "..", PE.max().round(2))

# 1) cómo se usa: embedding_final = embedding_token + PE[posición]
# X (n, d) son los embeddings de los tokens; se suma la posición:
# X_pos = X + PE[:len(X)]

# 2) propiedad clave: el producto punto entre posiciones decae suave con la distancia
sims = PE @ PE.T
print("PE[0]·PE[k] (k=0..5):", np.round(sims[0, :6], 1))

plt.figure(figsize=(8, 4))
plt.imshow(PE, cmap="RdBu", aspect="auto")
plt.xlabel("dimensión"); plt.ylabel("posición"); plt.colorbar()
plt.title("Codificación posicional sinusoidal"); plt.savefig("kata14_pe.png")
```

Salida esperada (forma)

```
PE shape: (100, 64) | rango: -1.0 .. 1.0
PE[0]·PE[k] (k=0..5): [32.  28.5 24.1 ...]   ← decae con la distancia
```

El mapa de calor muestra el patrón de franjas: dimensiones de baja frecuencia (cambian lento, codifican posición gruesa) a alta frecuencia (cambian rápido, posición fina).

Errores comunes / debugging

- **Concatenar en vez de sumar.** El Transformer original **suma** PE al embedding (misma dimensión `d`), no concatena.
- **Índice de frecuencia mal** (`2*(i//2)` vs `i`) → el emparejamiento seno/coseno se rompe.
- **Olvidar PE por completo** → el modelo trata la oración como bolsa de tokens; el orden se pierde (es el anti-patrón).
- **Reusar PE de longitud fija** con secuencias más largas que `max_len` (en versiones aprendidas, falla; en sinusoidal, extrapola).

Variantes y extensiones

1. Implementa **embeddings posicionales aprendidos** (una matriz entrenable) y discute el trade-off de extrapolación frente a la sinusoidal.
2. Demuestra empíricamente la propiedad de **desfase lineal**: verifica que existe una matriz `M_k` tal que `PE(pos+k) ≈ M_k·PE(pos)`.
3. Repite el experimento de permutación: pasa una secuencia y su versión barajada por la self-attention de la Kata 13 con y sin PE; confirma que **sin** PE la salida (reordenada) coincide y **con** PE no.

Anti-patrón

> ***Olvidar la codificación posicional*** *y esperar que la self-attention "deduzca" el orden. No puede: es permutación-invariante por construcción. Sin posición, el Transformer es un Bag-of-Words sofisticado y pierde toda la sintaxis.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
PE correcta	Forma `(max_len, d)`, valores en [-1,1], seno/coseno alternados
Uso correcto	Se **suma** al embedding de token, misma dimensión
Visualización	Mapa de calor de franjas multi-frecuencia guardado
Demostración	Muestras (permutación) que sin PE se pierde el orden

Reflexión

1. ¿Por qué se eligen funciones sinusoidales de frecuencias geométricamente espaciadas y no, p. ej., la posición entera normalizada?
2. ¿Qué ventaja da la PE sinusoidal frente a una aprendida cuando llegan secuencias más largas que las de entrenamiento?
3. La RNN codifica posición *implícitamente* por su recurrencia. ¿Qué ganamos y qué perdemos al hacerla *explícita* en el Transformer?

Referencias

- Vaswani et al. (2017), sección 3.5 (Positional Encoding).

- Su et al. (2021). *RoFormer: RoPE.* / Press et al. (2021). *ALiBi.* (variantes modernas.)

Kata 15 — Inspeccionar y usar un Transformer real (HuggingFace)

Objetivo. Cerrar el círculo: tomar un Transformer pre-entrenado (BERT/BETO), **extraer y visualizar sus pesos de atención reales**, y conectar lo construido a mano (katas 11–14) con la herramienta de producción que ya usaste en las katas 09–10.

Contexto y motivación

Ya implementaste atención, multi-head y posición. Ahora verifica que un modelo real hace exactamente eso: extrae las matrices de atención de BERT sobre una frase y observa qué aprenden las cabezas (algunas siguen la sintaxis, otras la puntuación, otras la correferencia). Esto convierte la teoría en algo palpable y te da una herramienta de diagnóstico/depuración para fine-tuning.

Teoría formal

Un encoder Transformer (BERT) apila `L` bloques idénticos; cada bloque:

```
Z   = LayerNorm( X + MultiHeadSelfAttention(X) )      # sub-capa 1 + residual
out = LayerNorm( Z + FFN(Z) )                          # sub-capa 2 + residual
FFN(z) = max(0, z·W₁ + b₁)·W₂ + b₂                     # feed-forward por posición
```

- **Conexión residual** (`X +`): deja pasar el gradiente sin atenuarse → entrena redes profundas (es la misma idea que la "autopista" de la celda LSTM, Kata 11).
- **LayerNorm**: estabiliza activaciones.
- BERT-base: `L=12` bloques, `h=12` cabezas, `d=768` → 12×12 = 144 matrices de atención por oración. Cada una es el `softmax(QKᵀ/√d_k)` de la Kata 13, con datos reales.

Implementación completa

```
from transformers import AutoTokenizer, AutoModel
import torch, matplotlib.pyplot as plt

NOMBRE = "dccuchile/bert-base-spanish-wwm-cased"        # BETO
tok = AutoTokenizer.from_pretrained(NOMBRE)
model = AutoModel.from_pretrained(NOMBRE, output_attentions=True).eval()

frase = "El gato que perseguía el perro era negro."
enc = tok(frase, return_tensors="pt")
with torch.no_grad():
    out = model(**enc)

attn = out.attentions                # tupla de L tensores (1, h, n, n)
print("nº de capas:", len(attn), "| forma por capa:", tuple(attn[0].shape))

tokens = tok.convert_ids_to_tokens(enc.input_ids[0])
capa, cabeza = 4, 7                  # elige una capa/cabeza
A = attn[capa][0, cabeza].numpy()    # (n, n)
print(f"\nCapa {capa} cabeza {cabeza}: fila de '[CLS]' suma {A[0].sum():.3f}")

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(A, cmap="viridis")
ax.set_xticks(range(len(tokens))); ax.set_xticklabels(tokens, rotation=90)
ax.set_yticks(range(len(tokens))); ax.set_yticklabels(tokens)
ax.set_title(f"Atención — capa {capa}, cabeza {cabeza}")
fig.colorbar(im); plt.tight_layout(); plt.savefig("kata15_attn.png")
```

Salida esperada (forma)

```
nº de capas: 12 | forma por capa: (1, 12, 11, 11)
Capa 4 cabeza 7: fila de '[CLS]' suma 1.000
```

El mapa de calor muestra qué token atiende a cuál. Explora varias cabezas: encontrarás cabezas que atienden al token siguiente, otras al `[SEP]`, otras que conectan el sujeto con su verbo o con su adjetivo (`gato` – `negro`). Cada fila suma 1: es el softmax de la Kata 13, real.

Errores comunes / debugging

- **No pasar** `output_attentions=True` → `out.attentions` es `None`.
- **Olvidar** `[CLS]` / `[SEP]` al etiquetar ejes → los índices se corren.
- **Interpretar una sola cabeza como "lo que entiende BERT".** Son 144 matrices; el comportamiento es distribuido. (Mismo matiz que el anti-patrón de la Kata 12.)
- **Subtokens** `##` parten palabras → la matriz tiene más filas que palabras.

Variantes y extensiones

1. **Promedia las cabezas** de una capa y compáralo con cabezas individuales: la media oculta los roles especializados.

2. Usa `bertviz` para una visualización interactiva de las 144 cabezas.
3. **Fine-tuning mínimo:** añade una cabeza de clasificación sobre `[CLS]` y afina BETO en el dataset de la Kata 08; compara su F1 contra TF-IDF+LogReg. Cierra el arco clásico→neuronal.
4. Reconstruye un **bloque encoder a mano** (atención de la Kata 13 + FFN + residual + LayerNorm) y verifica que, con los pesos de BERT, reproduces su salida de la primera capa.

Anti-patrón

> ***Usar Transformers como caja negra sin entender su costo ni su mecanismo.*** *Sin saber que la atención es $O(n^2)$, que necesita posición explícita, o que las cabezas se especializan, tomas malas decisiones: secuencias innecesariamente largas, prompts mal estructurados, fine-tuning a ciegas. Las katas 11–14 existen para que esta caja no sea negra.*

Criterio de aceptación (rúbrica)

Punto	Evidencia exigida
Atenciones extraídas	Forma `(L, h, n, n)` reportada; filas suman 1
Visualización	Mapa de calor etiquetado con tokens, guardado
Interpretación	Identificas al menos una cabeza con un rol reconocible
Conexión	Relacionas explícitamente con tu self-attention de la Kata 13

Reflexión

1. ¿Por qué las conexiones residuales son indispensables para entrenar 12+ capas? (relación con el vanishing gradient de la Kata 11.)
2. ¿Qué evidencia ves de que distintas cabezas aprenden funciones distintas?
3. Recorre el arco completo: ¿cómo se conecta el tokenizador (Kata 02), la posición (Kata 14), la self-attention (Kata 13) y el embedding contextual (Kata 09) dentro de **una sola** pasada de BERT?

Referencias

- Vaswani et al. (2017). *Attention Is All You Need.*
- Devlin et al. (2018). *BERT.*
- Clark et al. (2019). *What Does BERT Look At? An Analysis of BERT's Attention.*
- Vig (2019). *bertviz* (herramienta de visualización).

Cierre — Mapa de competencias y orden sugerido

Kata	Competencia	Etapa del continuum	Notebook / recurso base
01	BPE desde cero	Tokenización	`tokenizacion/Sentencepiece+bpe.ipynb`
02	Comparar tokenizadores (SentencePiece)	Tokenización	`tokenizacion/act4-tokenizadores.ipynb`
03	TF-IDF + coseno (Shakespeare)	Repr. dispersa	`semantica/act6-semantica.ipynb`
04	Normalización L2	Repr. dispersa	—
05	PCA vs t-SNE	Repr. dispersa	`semantica/*.png`
06	Word2Vec skip-gram + neg. sampling	Embeddings densos	`semantica/Word2vec_scratch.ipynb`
07	GloVe vs fastText (OOV)	Embeddings densos	—
08	TF-IDF vs embeddings (clasificación)	Aplicación	`semantica/regresi_n_logistica_b_sica_2.ipynb`
09	Polisemia BERT/BETO	Contextual	`embeddings/taller7_solucion.ipynb`
10	Retrieval SBERT/BioBERT	Oración	`embeddings/Biobert`
11	RNN/LSTM char desde cero	Arquitectura (predecesor)	—
12	Atención Bahdanau/Luong	Arquitectura (puente)	—
13	Self-attention + multi-head	Arquitectura (Transformer)	—
14	Codificación posicional	Arquitectura (Transformer)	—

Kata	Competencia	Etapa del continuum	Notebook / recurso base
15	Inspeccionar Transformer real	Arquitectura (HuggingFace)	`embeddings/taller7_solucion.ipynb`

> ***Nota de orden.*** *Los Bloques A–D recorren las representaciones (de tokens a oraciones). El Bloque E baja un nivel: la arquitectura que las produce. Conceptualmente las katas 11–14 están* ***debajo*** *de las katas 09–10 (BERT/SBERT son Transformers). Puedes hacerlas en orden lineal (11→15 al final, como "cómo funciona por dentro") o intercalarlas justo antes de la Kata 09 si prefieres entender el motor antes de usarlo.*

El hilo conductor (para la reflexión final)

Katas	Pregunta	Respuesta
01–02	¿Cómo **partir** el texto?	unidades discretas (tokens)
03–05	¿Cómo **comparar** documentos?	vectores dispersos + geometría (coseno)
06–08	¿Cómo **codificar** significado?	vectores densos aprendidos (estáticos)
09	¿Cómo distinguir **sentido**?	vectores contextuales (dependen de la frase)
10	¿Cómo **recuperar** por sentido?	vectores de oración + búsqueda
11–15	¿Qué **arquitectura** produce todo esto?	RNN/LSTM → atención → Transformer

Cada flecha resuelve la limitación de la etapa anterior: el conteo crudo no compara bien → TF-IDF+coseno; lo disperso no captura sinónimos → embeddings densos; lo estático no distingue sentidos → contextuales; los de token no comparan oraciones → SBERT. Entender **por qué** cada etapa existe vale más que dominar cualquiera de ellas aislada.

Criterio global de "kata terminada"

Una kata está completa cuando se cumplen **las cuatro**:

1. **Corre.** El notebook/script ejecuta de principio a fin sin error.
2. **Se entiende a mano.** Puedes resolver el ejemplo numérico minúsculo con lápiz.
3. **Está justificada.** Explicas el *porqué* del output, no solo el output.
4. **Evita el anti-patrón de forma demostrable.** Tienes el experimento que prueba por qué el camino correcto es mejor que el incorrecto.

> *Si te falta cualquiera de las cuatro, es media kata. Repite hasta que salga fluido.*